

gnn4s 开发手册

介绍

安装

依赖

圆柱绕流仿真

模型与算法

代码

文件结构

主要流程

参数设置

读取数据

搭建网络

训练网络

预测和评价

后处理可视化

旗帜运动仿真

模型与算法

代码

文件结构

主要流程

参数设置

读取数据

搭建网络

训练网络

预测和评价

后处理可视化

机翼仿真

数据集

代码

文件结构

主要流程

参数设置

读取数据

搭建网络

训练网络

预测和评价

后处理可视化

心血管血流模拟

算法与模型

一维模型

节点特征和边特征

多步预测

代码

文件结构

主要流程

参数设置

读取数据

搭建网络

训练网络

预测和评价

后处理可视化

分子构象生成

模型与算法

代码

- 文件结构
- 主要流程
- 参数设置
- 读取数据
- 搭建网络
- 扩散过程
- 损失函数
- 训练网络
- 预测
- 后处理可视化
- 评价

分子对接

- 模型与算法
 - 算法原理
 - 配体构象变换
 - 乘积空间上的扩散
- 模型架构

代码

- 文件结构
- 主要流程
- 参数设置
- 读取数据
- 搭建网络
- 损失函数
- 训练网络

gnn4s 开发手册

介绍

gnn4s (Graph Neural Network for Science) 是一个基于图神经网络的软件框架, 旨在解决复杂的科学和工程问题. 该框架集成了多种先进的图神经网络模型, 为工业制造, 材料设计和药物发现等应用领域中的一系列具有挑战性的问题提供了有力的解决方案.

gnn4s 提供了灵活且高效的工具, 使用户能够更轻松地搭建和训练模型, 从而深入研究各种复杂的物理化学现象. 其用户友好的接口和丰富的功能集, 使得在实际应用中进行图神经网络实验变得更加容易.

gnn4s 正在不断更新和优化, 以适应新的科学发展和工程需求, 为科研人员和工程师提供了一个强大而灵活的工具, 帮助他们更好地理解 and 解决实际问题.

安装

克隆仓库:

```
git clone git@github.com:AI4SciFoundation/gnn4s.git
cd gnn4s
```

创建一个包含依赖项的虚拟环境并激活它:

```
conda env create -n gnn4s python==3.10
conda activate gnn4s
chmod +x install_dependencies.sh
./install_dependencies.sh
```

依赖

目前gnn4s依赖pytorch和torch-geometric处理图数据, 搭建和训练图神经网络, 依赖biopython和rdkit等package处理化学和生物信息数据.

```
torch==2.1.0+cu118
torch-scatter==2.1.2
torch-sparse==0.6.18
torch-cluster==1.6.3
torch-geometric==2.4.0
biopython==1.82
easydict==1.11
e3nn==0.5.1
molsets==0.1.3
pandas==2.1.4
pyemd==1.0.0
pyyaml==6.0.1
rdkit==2023.9.3
```

圆柱绕流仿真

在这一章节中, 我们将详细介绍使用gnn4s搭建图神经网络MeshGraphNet[], 并用于仿真2D情形下的圆柱绕流. 通过这个例子, 您将对gnn4s的基本框架和应用方式有一些初步的了解.

模型与算法

MeshGraphNet使用模拟网格 $M^t = (V, E^M)$ 描述系统在时间 t 的状态, 其中节点 V 由网格边 E^M 连接. 每个节点 $i \in V$ 与一个参考网格空间坐标 u^i 以及需要建模的动态量 q_i 相关联. 在这个算例中, 动态量 q_i 为流体的速度. 任务是在给定当前网格 M^t 和动态量的情况下, 学习网格在时间 $t + 1$ 的动态量. 在这个算例中, 网格坐标固定, 不随时间发生变化, 采用欧拉(Euler)系统对网格上速度场的变化进行建模(若网格坐标随时间变化, 则采用拉格朗日(Lagrange)系统建模, 相关应用将在下一个章节中介绍).

编码器 编码器将当前网格 M^t 编码为图 $G = (V, E^M)$. 网格节点在图中变成图节点 V , 网格边在图中变成双向网格边 E^M . 然后将网格空间中的相对位移向量 $u_{ij} = u_i - u_j$ 及其范数 $|u_{ij}|$ 编码到网格边缘 $e_{ij}^M \in E^M$ 中, 从而将特征编码到图节点和边缘中. 所有剩余的动态特征 q_i 以及指示节点类型的一个热向量作为节点特征 v_i 提供.

最后使用MLP编码器 ϵ^M, ϵ^V 分别对网格边 e_{ij}^M 和节点 v_i 进行编码, 将上述连接特征编码为每个节点和边的大小为128的潜在向量.

处理器 通过以下方式将网格边 e_{ij}^M 和节点 v_i 嵌入分别更新为 $e_{ij}'^M$ 和 v_i'

$$e_{ij}'^M \leftarrow f^M(e_{ij}^M, v_i, v_j), \quad v_i' \leftarrow f^V(v_i, \sum_j e_{ij}^M),$$

其中 f^M, f^V 是使用残差连接的MLP.

解码器和状态更新器 为从时间 t 的输入预测时间 $t + 1$ 的状态, 解码器使用MLP δ^V 将最终处理步骤后的潜在节点特征 v_i 转换为输出特征 p_i . 输出特征 p_i 被解释为动态量 q_i 的导数, 并使用具有 $\Delta t = 1$ 的前向欧拉积分器进行积分, 以计算下一个步骤的动力学量 q_i^{t+1} . 积分方式为 $q_i^{t+1} = p_i + q_i^t$. 最后, 使用 q_i^{t+1} 更新输出网格节点 V 以产生 M^{t+1} .

代码

位于 `./demo/cylinder_flow` 中, 包含训练脚本 `cylinder_flow_train.py` 和推理脚本 `cylinder_flow_infer.py`.

文件结构

```
|--dataset                                # 数据集文件
|   |--cylinder_flow
|       |--raw                            # 原始数据
|           |--cylinder_flow_train.npy    # 训练所用的数据
|           |--cylinder_flow_valid.npy    # 推理所用的数据
|--checkpoint                            # 检查点
|   |--cylinder_flow
|       |--train                          # 训练保存的模型和相关参数设置
|       |--infer                          # 推理所用的模型和相关参数设置
|           |--cylinder_flow.pth          # 参考模型和相关参数设置
|--gnn4s                                 # gnn4s库
|   |--data                              # 数据类
|       |--dataset.py
|       |--load.py
|       |--process.py
```

```

|--mask.py
|--model                                # 网络模型
|--mesh_graph_net.py
|--mlp.py
|--normalization.py
|--train                                # 训练
|--loss.py
|--trainer.py
|--infer                                # 推理
|--predictor.py
|--evaluation                            # 评价
|--evaluator.py
|--postprocess                           # 后处理可视化
|--visualize.py
|--__init__.py
|--parser.py                            # 参数
|--config                               # 参数配置文件
|--cylinder_flow_train.yaml             # 训练参数
|--cylinder_flow_infer.yaml             # 推理参数
|--cylinder_flow_train.py               # 训练脚本
|--cylinder_flow_infer.py               # 推理脚本

```

主要流程

我们首先介绍训练和推理的主要流程, 然后再详细介绍每个部分.

训练 在训练脚本 `cylinder_flow_train.py` 中, 首先读取参数设置, 并加载日志.

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/cylinder_flow_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器.

```

# mask
node_type_list = {'NORMAL': 0, 'OBSTACLE': 1, 'AIRFOIL': 2, 'HANDLE': 3,
                  'INFLOW': 4, 'OUTFLOW': 5, 'WALL_BOUNDARY': 6, 'SIZE': 9}
mask_fn = gnn4s.data.NodeTypeMask(node_type_list, config.data.mask_node_type)
mask_noise_fn = gnn4s.data.NodeTypeMask(node_type_list,
                                         config.data.mask_noise_node_type)

# dataset
keys = ['mesh_pos', 'cells', 'node_type', 'velocity']
load_fn = gnn4s.utils.partial(gnn4s.data.load_npy, keys=keys)
process_fn = gnn4s.utils.partial(gnn4s.data.euler_lagrange_to_pyg_data,
                                 sys_type='euler', mask_fn=mask_fn,
                                 mask_noise_fn=mask_noise_fn, noise_std=config.data.noise_std,
                                 noise_gamma=config.data.noise_gamma, label_update=True)
dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,

```

```
load_fn=load_fn, process_fn=process_fn, split_time=True)

# dataloader
dataloader = gnn4s.data.DataLoader(dataset=dataset,
                                   batch_size=config.data.batch_size)
```

搭建MeshGraphNet网络.

```
# model
model = gnn4s.model.MeshGraphNet(node_input_dim=config.model.node_input_dim,
                                   edge_input_dim=config.model.edge_input_dim,
                                   output_dim=config.model.output_dim,
                                   block_num=config.model.block_num)

model.to(config.device)
```

构造均方损失函数.

```
# loss
loss = gnn4s.train.LossMS(model)
```

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```
# train
optimizer = gnn4s.train.AdamOptimizer(model.parameters(), lr=config.train.lr)
trainer = gnn4s.train.Trainer(train_dataloader=dataloader, model=model,
                               loss=loss, optimizer=optimizer, epoch_num=config.train.epoch_num,
                               ckpt_dir=config.ckpt.dir, ckpt_name=config.ckpt.name,
                               logger=logger, device=config.device)
trainer.train()
```

以上是训练的整个流程.

推理 在推理脚本 `cylinder_flow_infer.py` 中, 首先读取参数设置和checkpoint.

```
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/cylinder_flow_infer.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

    # load checkpoint
    ckpt_dict =
    gnn4s.utils.load_ckpt(f'{config.ckpt.dir}/{config.ckpt.name}.pth')
    ckpt_config = ckpt_dict['config']
    logger.log_config(vars(ckpt_config))
```

然后读取数据, 对数据进行一些处理, 转为图数据格式, 生成数据加载器.

```
# mask
```

```

node_type_list = {'NORMAL': 0, 'OBSTACLE': 1, 'AIRFOIL': 2, 'HANDLE': 3,
                  'INFLOW': 4, 'OUTFLOW': 5, 'WALL_BOUNDARY': 6, 'SIZE': 9}
mask_fn = gnn4s.data.NodeTypeMask(node_type_list, config.data.node_type)

# datasets
keys = ['mesh_pos', 'cells', 'node_type', 'velocity']
load_fn = gnn4s.utils.partial(gnn4s.data.load_npy, keys=keys)
process_fn = gnn4s.utils.partial(gnn4s.data.euler_lagrange_to_pyg_data,
                                 sys_type='euler', mask_fn=mask_fn)
dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                              load_fn=load_fn, process_fn=process_fn)

# dataloader
dataloader = gnn4s.data.DataLoader(dataset=dataset, shuffle=False)

```

搭建MeshGraphNet网络, 并读取模型参数.

```

# model
model =
gnn4s.model.MeshGraphNet(node_input_dim=ckpt_config.model.node_input_dim,

edge_input_dim=ckpt_config.model.edge_input_dim,
                    output_dim=ckpt_config.model.output_dim,
                    block_num=ckpt_config.model.block_num)
model.load_state_dict(ckpt_dict['model_state_dict'], config.device)

```

构造预测器和评价器, 预测解, 并评价预测解的误差.

```

# evaluator
predictor = gnn4s.infer.PredictorEuler(model, config.device)
evaluator = gnn4s.evaluation.Evaluator(dataloader, predictor)

# evaluate
error, pred = evaluator.evaluate()
for i in range(len(error)):
    logger.log_info(error[i])

```

最后进行后处理, 用预测解作图.

```

## postprocess
for data in dataloader:
    gnn4s.postprocess.plot_anime_trimesh(data, pred[0], config.output.name,
                                          config.output.dirt,
                                          dpi=config.output.dpi)

    break

```

以上是推理的整个流程.

参数设置

训练脚本中的相关部分:

```
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/cylinder_flow_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)
    ...
```

推理时还需要从checkpoint中读取部分参数设置

```
def main():
    ...
    # load checkpoint
    ckpt_dict =
    gnn4s.utils.load_ckpt(f'{config.ckpt.dir}/{config.ckpt.name}.pth')
    ckpt_config = ckpt_dict['config']
    logger.log_config(vars(ckpt_config))
    ...
```

训练所用的参数设置存放在 ./config/cylinder_flow_train.yaml 中:

```
data:
    # name of the dataset
    name: cylinder_flow_train
    # directory for storing the dataset
    dir: ./dataset/cylinder_flow/raw
    # standard deviation of noise
    noise_std: 0.02
    # noise parameter gamma
    noise_gamma: 1.0
    # the node types that apply a mask
    mask_node_type:
        - NORMAL
        - OUTFLOW
    # the node types that apply a noise mask
    mask_noise_node_type:
        - NORMAL
    # batch size
    batch_size: 1

model:
    # dimension of node input features
    node_input_dim: 11
    # dimension of edge input features
    edge_input_dim: 3
    # dimension of output
    output_dim: 2
```



```

# number of hidden block
block_num: 10

train:
  # learning rate
  lr: 0.0001
  # number of epochs
  epoch_num: 1

ckpt:
  # name of checkpoint
  name: cylinder_flow
  # directory for storing checkpoint
  dirt: ./checkpoint/cylinder_flow/train

log:
  # directory for saving log
  dirt: ./log/cylinder_flow

# computing device
device: cuda

```

推理所用的参数设置存放在 `./config/cylinder_flow_infer.yaml` 中

```

data:
  # name of the dataset
  name: cylinder_flow_valid
  # directory for storing the dataset
  dirt: ./dataset/cylinder_flow/raw
  # the node types that apply a mask
  node_type:
    - NORMAL
    - OUTFLOW
  # batch size
  batch_size: 1

ckpt:
  # name of checkpoint
  name: cylinder_flow
  # directory for storing checkpoint
  dirt: ./checkpoint/cylinder_flow/infer

output:
  # name of the output file
  name: cylinder_flow
  # directory for storing the output file
  dirt: ./dataset/cylinder_flow/output
  # output image resolution.
  dpi: 100

log:
  # directory for saving log
  dirt: ./log/cylinder_flow

# computing device

```

```
device: cuda
```

读取数据

训练脚本中的相关部分:

```
def main():
    ...
    # mask
    node_type_list = {'NORMAL': 0, 'OBSTACLE': 1, 'AIRFOIL': 2, 'HANDLE': 3,
                      'INFLOW': 4, 'OUTFLOW': 5, 'WALL_BOUNDARY': 6, 'SIZE': 9}
    mask_fn = gnn4s.data.NodeTypeMask(node_type_list, config.data.mask_node_type)
    mask_noise_fn = gnn4s.data.NodeTypeMask(node_type_list,
                                             config.data.mask_noise_node_type)

    # dataset
    keys = ['mesh_pos', 'cells', 'node_type', 'velocity']
    load_fn = gnn4s.utils.partial(gnn4s.data.load_npy, keys=keys)
    process_fn = gnn4s.utils.partial(gnn4s.data.euler_lagrange_to_pyg_data,
                                     sys_type='euler', mask_fn=mask_fn,
                                     mask_noise_fn=mask_noise_fn, noise_std=config.data.noise_std,
                                     noise_gamma=config.data.noise_gamma, label_update=True)
    dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                                 load_fn=load_fn, process_fn=process_fn, split_time=True)

    # dataloader
    dataloader = gnn4s.data.DataLoader(dataset=dataset,
                                       batch_size=config.data.batch_size)

    ...
```

注意推理时不需要给数据添加噪声.

数据类 `Dataset` 位于 `./gnn4s/data/dataset.py` 中, 继承于 `torch_geometric.data.Dataset`.

`Dataset` 提供了一个方便的接口, 用于加载, 处理和使用图数据集. 可以使用这个类来创建自定义的图数据集, 并在训练神经网络时使用.

`Dataset` 的输入 `load_fn` 用于读取数据, `process_fn` 用于将读取的原始数据转换为图数据 `torch_geometric.data.Data`, 以用于后续的训练. 图数据由节点特征矩阵, 边索引和目标标签组成.

```
from torch_geometric.data import Dataset as PyGDataset

class Dataset(PyGDataset):
    """ graph data class """
    def __init__(self, name, dirt, load_fn, process_fn, normalizer=None,
                 split_time=False):
        super(Dataset, self).__init__()
        """ initialization
        Args:
            name: name of the dataset
            dirt: directory for storing the dataset
            load_fn: function for loading the dataset
            process_fn: function for processing the dataset
            normalizer: class for normalize data
            split_time: whether to split data according to time
        """
```

```

# load data
self.data_list = self.load_fn(self.dirt, self.name)

# process data
data_list = []
with tqdm(total=len(self.data_list), desc='processing data') as pbar:
    for data in map(self.process_fn, self.data_list):
        if not self.split_time:
            data_list.append(data)
        else:
            data_list += data
        pbar.update()
self.data_list = data_list

# normalize data
if self.normalizer is not None:
    ...
    with tqdm(total=len(self.data_list), desc='normalizing data') as
pbar:
        for data in data_list:
            self.normalizer.normalize(data)
            pbar.update()

def len(self):
    """ return the number of graph data in the dataset """
    ...
def get(self, idx):
    """ retrieve an individual graph data based on the index """
    ...

```

读取数据的函数 `gnn4s.data.load_npy` 位于 `./gnn4s/data/load.py` 中, 功能是从 `.npy` 文件中按照给定的 `keys` 列表读取需要的数据.

```

def load_npy(name: str, dirt: str, keys: list):
    """ load .npy format dataset
    Args:
        name: name of dataset
        dirt: directory storing dataset
        keys: keys of data to be loaded
    Returns:
        data_list: list of dataset
    """
    path = f'{dirt}/{name}.npy'
    files = np.load(path, allow_pickle=True)

    data_list = []
    for fi in range(len(files)):
        data = {}
        for k in keys:
            data[k] = files[fi][k]
        data_list.append(data)

    return data_list

```

在这个应用中, keys 列表为 ['mesh_pos', 'cells', 'node_type', 'velocity'], `gnn4s.data.load_numpy` 将遍历数据文件中的所有数据, 并按照 keys 从每条数据中读取需要的数据.

预处理数据的函数 `gnn4s.data.euler_lagrange_to_pyg_data` 位于 `./gnn4s/data/process.py` 中, 功能是对上一步读取的数据进行一些预处理, 转换为 `PyGData`.

`PyGData(torch_geometric.data.Data)` 是 `torch_geometric` 库中的图数据结构. 在这个应用中, `PyGData` 包含的属性主要有边索引 `edge_index`, 节点网格坐标 `mesh_pos`, 节点特征 `x`, 节点标签 `y`, 和节点遮盖 `mask`. 由于这个应用所采用的动力系统为 Euler 系统, 不涉及节点世界坐标 `world_pos`, 相关内容将在下一章中介绍.

函数 `gnn4s.data.euler_lagrange_to_pyg_data` 将原始空间中的网格面索引转换为图神经网络接受的输入边索引, 将节点类型和节点当前时刻的速度合并为节点特征, 将节点下一时刻的速度作为节点标签, 并在节点特征和标签中加入噪声. 由于训练任务是学习当前时刻到下一时刻速度的变化量, 所以训练数据中的标签会被进一步修改. 最后, 根据节点的类型生成相应的 `mask`.

```
def euler_lagrange_to_pygdata(data, sys_type, mask_fn, mask_noise_fn=None,
                              noise_std: float=None, noise_gamma: float=None,
                              add_noise: bool=False, label_update: bool=False,
                              dtype=torch.float32):
    """ transform numpy data to pygdata
    Args:
        data: numpy data
        sys_type: type of dynamical system, euler or lagrange
        mask_fn: mask adding to computing node
        mask_noise_fn: mask adding to node requiring noise addition
        noise_std: standard deviation of the noise distribution
        noise_gamma: ratio of noise
        add_noise: whether to add noise
        label_update: whether the label is an update quantity
    """
    pyg_data = []
    time_step_num = len(data['mesh_pos'])
    for i in range(1, time_step_num-1):
        # convert face index to edge index
        face_index = torch.as_tensor(data['cells'][i].T, dtype=torch.long)
        edge_index = face_to_edge(face_index)

        # mesh_pos, world_pos and velocity
        mesh_pos = torch.as_tensor(data['mesh_pos'][i], dtype=dtype)
        if sys_type=='euler':
            world_pos, world_pos_prev = None, None
            velocity = torch.as_tensor(data['velocity'][i], dtype=dtype)
        if sys_type=='lagrange':
            ...

        # node attribute and label
        node_type = torch.as_tensor(data['node_type'][i], dtype=dtype)
        x = velocity
        if sys_type=='euler':
            y = torch.as_tensor(data['velocity'][i+1], dtype=dtype)
        if sys_type=='lagrange':
            ...

        # add noise
```

```

if add_noise:
    noise = torch.normal(mean=0.0, std=noise_std, size=x.shape)
    mask_noise = mask_noise_fn(node_type[:,0])
    noise[~mask_noise] = 0
    x += noise
    y += (1.0-noise_gamma) * noise

# calculate the update quantity (if need)
if label_update:
    if sys_type=='euler':
        y = y - x
    if sys_type=='lagrange':
        ...
x = torch.cat([node_type,x],1)

# transform to pygdata
mask = mask_fn(node_type[:,0])
pyg_data.append(PyGData(edge_index=edge_index, face_index=face_index,
                        mesh_pos=mesh_pos, world_pos=world_pos,
                        world_pos_prev=world_pos_prev, x=x, y=y,
                        mask=mask))
return pyg_data

```

生成数据类 Dataset 后, 将数据类 Dataset 将传递给 DataLoader, 用于加载图数据集. 它会负责处理数据的批处理, 随机采样等操作.

```

from torch_geometric.loader import DataLoader as PyGDataLoader

class DataLoader(PyGDataLoader):
    """ dataLoader for loading graph dataset """
    def __init__(self, dataset, batch_size: int=1, shuffle: bool=True):
        super(DataLoader, self).__init__(dataset, batch_size, shuffle)
    """ initialization
    Args:
        dataset: dataset class
        batch_size: size of batch data
        shuffle: whether to shuffle the data
    """

```

搭建网络

训练脚本中的相关部分:

```

def main():
    ...
    # model
    model = gnn4s.model.MeshGraphNet(node_input_dim=config.model.node_input_dim,
                                     edge_input_dim=config.model.edge_input_dim,
                                     output_dim=config.model.output_dim,
                                     block_num=config.model.block_num)

    model.to(config.device)
    ...

```

推理时调用 model.load_state_dict() 读取经过训练的模型.

MeshGraphNet 由编码器, 隐藏层和解码器构成.

```
class MeshGraphNet(nn.Module):
    """ MeshGraphNet model """
    def __init__(self, node_input_dim: int=1, edge_input_dim: int=2, output_dim:
int=2,
                hidden_dim: int=128, block_num: int=1) -> None:
        super(MeshGraphNet, self).__init__()
        """ initialization
        Args:
            block_num: number of hidden block
            node_input_dim: dimension of node input features
            edge_input_dim: dimension of edge input features
            output_dim: dimension of output
        """
        ...

    def forward(self, graph: PyGData):
        """ Forward propogation
        Args:
            graph: graph data
        """
        ...
```

训练网络

训练脚本中的相关部分:

```
def main():
    ...
    # loss
    loss = gnn4s.train.LossMS(model)

    # train
    optimizer = gnn4s.train.AdamOptimizer(model.parameters(), lr=config.train.lr)
    trainer = gnn4s.train.Trainer(train_data_loader=data_loader, model=model,
        loss=loss, optimizer=optimizer, epoch_num=config.train.epoch_num,
        ckpt_dir=config.ckpt.dir, ckpt_name=config.ckpt.name,
        logger=logger, device=config.device)
    trainer.train()
```

在这个算例中, 我们采用均方损失函数度量预测解和真解之间的偏差.

```
class LossMS():
    """ mean squared loss """
    def __init__(self, model) -> None:
        """ initialization
        Args:
            model: model
        """
        self.model = model

    def __call__(self, batch_data) -> dict:
        """ calculate the value of the loss function
```

```

Args:
    batch_data: batch data
Returns:
    value of the loss function
"""
res = self.model(batch_data) - batch_data.y

mask = batch_data.mask
loss = (res[mask]**2).mean()
return {'loss': loss}

```

训练器负责管理整个训练流程。训练器中不仅集成了之前定义的数据加载器, 模型和损失函数, 也集成了多种训练机制, 包含学习率预热(learning rate warm-up), 梯度裁剪(Gradient clipping)和指数滑动平均(Exponential moving average)。用户可以根据具体需要调用。

```

class Trainer():
    """ Trainer for training model """
    def __init__(self, train_data_loader=None, valid_data_loader=None,
                  model=None, loss=None, error=None,
                  optimizer=None, lr: float=0.001, scheduler=None,
                  epoch_num: int=10000, warmup_step: int=0, grad_clip: float=0.0,
                  ema_rate: float=0.9999, use_ema: bool=False,
                  print_interval: int=1, save_interval: int=1, save_dir:
str=None,
                  ckpt_name: str=None, ckpt_dir: str=None, logger=None,
                  device='cpu') -> None:
    """ initialization
    Args:
        train_data_loader: train data loader
        valid_data_loader: valid data loader
        model: model
        loss: loss function
        error: error function
        optimizer: optimizer for minimizing the loss function
        lr: learning rate
        scheduler: learning rate scheduler
        epoch_num: number of epochs
        warmup_step: number of warmup steps for learning rate
        grad_clip: maximum norm of gradients
        ema_rate: smoothing coefficient
        use_ema: whether to use Exponential Moving Average
        print_interval: interval for print
        save_interval: interval for saving the model
        save_dir: directory for saving the data
        ckpt_name: name of the checkpoint
        ckpt_dir: directory for saving the checkpoint
        logger: logger
        device: computing device
    """
    ...
    def step(self, batch_data) -> torch.Tensor:
        """ Running one step of training
        Args:
            batch_data: batch data
        Returns:

```

```

        loss: value of loss function after training
    """

    def train(self):
        """ Train model """
        # loop
        while self.current_epoch < self.epoch_num:
            for batch_data in self.data_loader:
                loss = self.step(batch_data)
            ...

```

预测和评价

推理脚本中的相关部分:

```

def main():
    ...
    # evaluator
    predictor = gnn4s.infer.PredictorEuler(model, config.device)
    evaluator = gnn4s.evaluation.Evaluator(data_loader, predictor)

    # evaluate
    error, pred = evaluator.evaluate()
    for i in range(len(error)):
        logger.log_info(error[i])
    ...

```

预测器需要利用训练好的模型, 根据给定的初始状态预测所需时间步内的状态. 采用时间演进的方式, 将当前时刻的状态输入模型, 得到下一个时刻的状态预测. 循环往复, 直至终止时刻.

```

class PredictorEuler():
    """ predictor of euler system """
    def __init__(self, model, device):
        """ initialization
        Args:
            model: model
            device: computing device
        """
        ...

    def __call__(self, data):
        """ Predict the solution from the initial time step to the final time
step
        Args:
            data: data
        """
        predict = []

        vel_cur = data[0].x[:, 1:]
        for step in range(len(data)):
            # copy graph
            g = copy.deepcopy(data[step])
            if step > 0:
                g.x[:, 1:] = vel_next
                vel_cur = vel_next

```


可视化函数plot_anime_trimesh负责在三角网格上作出给定时刻的预测解的图像, 并将这些图像拼接为动画.

```
def plot_anime_trimesh(data, pred, name: str, dirt: str,
                       skip: int=10, interval: int=100, dpi: int=100) -> None:
    """ plot predict value on trimesh and create animation.
    Args:
        data: data
        pred: predict value
        name: name of the image
        dirt: directory for storing the image
        skip: number of skipping time step
        interval: time interval between every two frames
        dpi: resolution of image
    """
    ...
```

预测结果

旗帜运动仿真

在这一章节中, 我们将介绍如何使用gnn4s实现图神经网络MeshGraphNet的另一个应用: 模拟风中飘动的旗帜. 与上一个应用中所采用建模的欧拉系统不同, 在这个应用中将采用拉格朗日系统进行建模. 在欧拉系统中, 观察者固定在空间中的某一点, 关注物体在该点上的性质随时间的变化. 坐标系是固定的, 通常选择惯性坐标系. 而在拉格朗日系统中, 观察者随着物体移动, 关注物体在其运动轨迹上的性质的演变. 坐标系随着物体的移动而变化, 通常选择与物体一同运动的坐标系. 这两个框架提供了不同的视角, 适用于不同类型的物理问题.

模型与算法

仍然使用模拟网格 $M^t = (V, E^M)$ 描述系统在时间 t 的状态, 其中节点 V 由网格边 E^M 连接. 网格点相关的性质除了固定的网格空间坐标 u_i 外, 还包含额外的世界空间坐标 x_i , 用于描述网格在三维空间中的动态状态. 任务是在给定当前网格 M^t 和(可选)先前网格的历史 $\{M^{t-1}, \dots, M^{t-h}\}$ 的情况下, 学习网格在时间 $t + 1$ 的位置和动态量.

编码器 编码器将当前网格 M^t 编码为多重图 $G = (V, E^M, E^W)$. 网格节点在图中变成图节点 V , 网格边在图中变成双向网格边 E^M , 用于计算网格的内部动力学. 对于拉格朗日系统, 将世界边 E^W 添加到图中, 以便学习外部动力学, 如(自)碰撞和接触. 给定最小网格边长度量级的固定半径 r_W , 满足 $|x_i - x_j| < r_W$ 且不属于网格中已连接的节点对为世界边. 这鼓励使用世界边缘在空间上接近但在网格空间中相距甚远的节点之间传递信息.

接下来将特征编码到图节点和边缘中. 将网格空间中的相对位移向量 $u_{ij} = u_i - u_j$ 及其范数 $|u_{ij}|$ 编码到网格边缘 $e_{ij}^M \in E^M$ 中. 然后将相对世界空间位移向量 x_{ij} 及其范数 $|x_{ij}|$ 编码到网格边缘 $e_{ij}^M \in E^M$ 和世界边缘 $e_{ij}^W \in E^W$ 中. 所有剩余的动态特征 q_i 以及指示节点类型的一个独热向量作为节点特征 v_i 提供.

最后使用MLP编码器 $\epsilon^M, \epsilon^W, \epsilon^V$ 分别对网格边 e_{ij}^M , 世界边 e_{ij}^W 和节点 v_i 进行编码, 将上述连接特征编码为每个节点和边的大小为128的潜在向量.

处理器 通过以下方式将网格边 e_{ij}^M , 世界边 e_{ij}^W 和节点 v_i 嵌入分别更新为 $e_{ij}^{'M}, e_{ij}^{'W}$ 和 v_i'

$$e_{ij}^{'M} \leftarrow f^M(e_{ij}^M, v_i, v_j), \quad e_{ij}^{'W} \leftarrow f^W(e_{ij}^W, v_i, v_j), \quad v_i' \leftarrow f^V(v_i, \sum_j e_{ij}^{'M}, \sum_j e_{ij}^{'W}),$$

其中 f^M, f^W, f^V 是使用残差连接的MLP.

解码器和状态更新器 为了从时间 t 的输入预测时间 $t + 1$ 的状态, 解码器使用MLP δ^V 将最终处理步骤后的潜在节点特征 v_i 转换为输出特征 p_i . 输出特征 p_i 被解释为动态量 q_i 的二阶导数(加速度), 并使用具有 $\Delta t = 1$ 的前向积分器进行积分, 以计算下一个步骤的动力学量 q_i^{t+1} . 不同于上一应用中采用的一阶系统, 这个应用中采用二阶系统, 需要积分两次, 计算方法为 $q_i^{t+1} = p_i + 2q_i^t - q_i^{t-1}$. 最后, 使用 q_i^{t+1} 更新输出网格节点 V 以产生 M^{t+1} .

代码

位于 `./demo/flag_simple` 中, 包含训练脚本 `flag_simple_train.py` 和推理脚本 `flag_simple_infer.py`.

文件结构

```
--dataset # 数据集文件
--flag_simple
--raw # 原始数据
--flag_simple_train.npy # 训练所用的数据
```

```

|--flag_simple_valid.npy      # 推理所用的数据
|--checkpoint                 # 检查点
|--flag_simple
|--train                     # 训练保存的模型和相关参数设置
|--infer                     # 推理所用的模型和相关参数设置
|--flag_simple.pth           # 参考模型和相关参数设置
|--gnn4s                     # gnn4s库
|--dataset                   # 数据类
|--dataset.py
|--load.py
|--process.py
|--mask.py
|--model                     # 网络模型
|--mesh_graph_net.py
|--mlp.py
|--normalization.py
|--train                     # 训练
|--loss.py
|--trainer.py
|--infer                     # 推理
|--predictor.py
|--evaluation                 # 评价
|--evaluator.py
|--postprocess               # 后处理可视化
|--visualize.py
|--__init__.py
|--parser.py                 # 参数
|--config                    # 参数配置文件
|--flag_simple_train.yaml    # 训练参数
|--flag_simple_infer.yaml    # 推理参数
|--flag_simple_train.py      # 训练脚本
|--flag_simple_infer.py      # 推理脚本

```

主要流程

训练 在训练脚本 `flag_simple_train.py` 中. 首先读取参数设置, 这一流程与在圆柱绕流仿真应用中一致.

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/flag_simple_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器. 这一流程与在圆柱绕流仿真应用中的主要不同在于数据处理函数 `gnn4s.data.euler_lagrange_to_pyg_data` 采用 `lagrange` 类型.

```

# mask
node_type_list = {'NORMAL': 0, 'OBSTACLE': 1, 'AIRFOIL': 2, 'HANDLE': 3,
                  'INFLOW': 4, 'OUTFLOW': 5, 'WALL_BOUNDARY': 6, 'SIZE': 9}

```

```

mask_fn = gnn4s.data.NodeTypeMask(node_type_list, config.data.mask_node_type)
mask_noise_fn = gnn4s.data.NodeTypeMask(node_type_list,
                                         config.data.mask_noise_node_type)

# dataset
kkeys = ['mesh_pos', 'world_pos', 'cells', 'node_type']
load_fn = gnn4s.utils.partial(gnn4s.data.load_npy, keys=kkeys)
process_fn = gnn4s.utils.partial(gnn4s.data.euler_lagrange_to_pyg_data,
                                 sys_type='lagrange', mask_fn=mask_fn,
                                 mask_noise_fn=mask_noise_fn, noise_std=config.data.noise_std,
                                 noise_gamma=config.data.noise_gamma, label_update=True)
dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                              load_fn=load_fn, process_fn=process_fn, split_time=True)

# dataloader
dataloader = gnn4s.data.DataLoader(dataset=dataset,
                                   batch_size=config.data.batch_size)

```

接下来的训练流程与在圆柱绕流仿真应用中一致. 搭建MeshGraphNet网络.

```

# model
model = gnn4s.model.MeshGraphNet(node_input_dim=config.model.node_input_dim,
                                  edge_input_dim=config.model.edge_input_dim,
                                  output_dim=config.model.output_dim,
                                  block_num=config.model.block_num)

model.to(config.device)

```

构造均方损失函数.

```

# loss
loss = gnn4s.train.LossMS(model)

```

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```

# train
optimizer = gnn4s.train.AdamOptimizer(model.parameters(), lr=config.train.lr)
trainer = gnn4s.train.Trainer(train_dataloader=dataloader, model=model,
                              loss=loss,
                              optimizer=optimizer,
                              epoch_num=config.train.epoch_num,
                              ckpt_dirt=config.ckpt.dirt,
                              ckpt_name=config.ckpt.name,
                              logger=logger, device=config.device)

trainer.train()

```

以上是训练的整个流程.

推理 在推理脚本 `flag_simple_infer.py` 中, 首先读取参数设置. 这一流程与在圆柱绕流仿真应用中一致.

```

import gnn4s

def main():

```

```

# configuration
config = gnn4s.load_config('./config/flag_simple_infer.yaml')

# logger
logger = gnn4s.utils.Logger(config.log.dirt)
logger.log_config(config)

# load checkpoint
ckpt_dict =
gnn4s.utils.load_ckpt(f'{config.ckpt.dirt}/{config.ckpt.name}.pth')
ckpt_config = ckpt_dict['config']
logger.log_config(vars(ckpt_config))

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器. 这一流程与在圆柱绕流仿真应用中的主要不同在于数据处理函数 `gnn4s.data.euler_lagrange_to_pyg_data` 采用 `lagrange` 类型.

```

# mask
node_type_list = {'NORMAL': 0, 'OBSTACLE': 1, 'AIRFOIL': 2, 'HANDLE': 3,
                  'INFLOW': 4, 'OUTFLOW': 5, 'WALL_BOUNDARY': 6, 'SIZE': 9}
mask_fn = gnn4s.data.NodeTypeMask(node_type_list, config.data.node_type)

# datasets
keys = ['mesh_pos', 'world_pos', 'cells', 'node_type']
load_fn = gnn4s.utils.partial(gnn4s.data.load_npy, keys=keys)
process_fn = gnn4s.utils.partial(gnn4s.data.euler_lagrange_to_pyg_data,
                                 sys_type='lagrange', mask_fn=mask_fn)
dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                             load_fn=load_fn, process_fn=process_fn)

# dataloader
dataloader = gnn4s.data.DataLoader(dataset=dataset, shuffle=False)

```

搭建 `MeshGraphNet` 网络, 并读取模型参数. 这一流程与在圆柱绕流仿真应用中一致.

```

# model
model =
gnn4s.model.MeshGraphNet(node_input_dim=ckpt_config.model.node_input_dim,
                          edge_input_dim=ckpt_config.model.edge_input_dim,
                          output_dim=ckpt_config.model.output_dim,
                          block_num=ckpt_config.model.block_num)
model.load_state_dict(ckpt_dict['model_state_dict'], config.device)

```

构造预测器和评价器, 预测解, 并评价预测解的误差. 这一流程与在圆柱绕流仿真应用中的主要不同在于预测器 `gnn4s.infer.PredictorLagrange` 采用 `lagrange` 类型.

```

# evaluator
predictor = gnn4s.infer.PredictorLagrange(model, config.device)
evaluator = gnn4s.evaluation.Evaluator(data_loader, predictor)

# evaluate
error, pred = evaluator.evaluate()
for i in range(len(error)):
    logger.log_info(error[i])

```

最后进行后处理, 用预测解作图.

以上是推理的整个流程.

参数设置

训练和推理脚本中的相关部分与在圆柱绕流仿真中一致.

训练所用的参数设置在 `./config/flag_simple_train.yaml` 中. 这与在圆柱绕流仿真中的主要不同在于噪声的尺度和模型的输入输出节点数目不同.

```

data:
  # name of the dataset
  name: flag_simple_train
  # directory for storing the dataset
  dirt: ./dataset/flag_simple/raw
  # standard deviation of noise
  noise_std: 0.0003
  # noise paramter gamma
  noise_gamma: 0.1
  # the node types that apply a mask
  mask_node_type:
    - NORMAL
  # the node types that apply a noise mask
  mask_noise_node_type:
    - NORMAL
  # batch size
  batch_size: 1

model:
  # number of hidden block
  block_num: 10
  # dimension of node input features
  node_input_dim: 12
  # dimension of edge input features
  edge_input_dim: 7
  # dimension of edge input features
  output_dim: 3

train:
  # learning rate
  lr: 0.0001
  # number of epochs

```

```

epoch_num: 1

ckpt:
  # name of checkpoint
  name: flag_simple
  # directory for storing checkpoint
  dirt: ./checkpoint/flag_simple/train

log:
  # directory for saving log
  dirt: ./log/cylinder_flow

# computing device
device: cuda

```

推理所用的参数设置在 `./config/flag_simple_infer.yaml` 中. 这与在圆柱绕流仿真中是类似的.

```

data:
  # name of the dataset
  name: flag_simple_valid
  # directory for storing the dataset
  dirt: ./dataset/flag_simple/raw
  # the node types that apply a mask
  node_type:
    - NORMAL

ckpt:
  # name of checkpoint
  name: flag_simple
  # directory for storing checkpoint
  dirt: ./checkpoint/flag_simple/infer

output:
  # name of the output file
  name: flag_simple
  # directory for storing the output file
  dirt: ./dataset/flag_simple/output
  # output image resolution.
  dpi: 150

log:
  # directory for saving log
  dirt: ./log/flag_simple

# computing device
device: cuda

```

读取数据

这一流程与在圆柱绕流仿真应用中类似. 一个区别在于读取数据时用于索引的 `keys` 列表有所不同. 在这个应用中, `keys = ['mesh_pos', 'world_pos', 'cells', 'node_type']`. 由于这个应用采用的动力系统是lagrange系统, 节点属性增加了世界坐标 `world_pos`, 相应地, 节点不直接具有属性 `velocity`, 而将根据当前时刻和下一时刻的世界坐标计算得到.

另一个区别在数据处理函数 `gnn4s.data.euler_lagrange_to_pyg_data` 采用 `lagrange` 类型. 与 `Euler` 类型不同, 节点属性中的速度将根据世界坐标计算得到, 而节点标签也由速度更换为世界坐标. 对于训练数据, 节点标签仍旧采用速度变化量, 但将根据上一时刻, 当前时刻和下一时刻计算得到.

```
def euler_lagrange_to_pygdata(data, sys_type, mask_fn, mask_noise_fn=None,
                              noise_std: float=None, noise_gamma: float=None,
                              add_noise: bool=False, label_update: bool=False,
                              dtype=torch.float32):
    """ Transform numpy data to pygdata
    Args:
        data: numpy data
        sys_type: type of dynamical system, euler or lagrange
        mask_fn: mask adding to computing node
        mask_noise_fn: mask adding to node requiring noise addition
        noise_std: standard deviation of the noise distribution
        noise_gamma: ratio of noise
        add_noise: whether to add noise
        label_update: whether the label is an update quantity
    """
    pyg_data = []
    time_step_num = len(data['mesh_pos'])
    for i in range(1, time_step_num-1):
        # convert face index to edge index
        ...

        # mesh_pos, world_pos and velocity
        mesh_pos = torch.as_tensor(data['mesh_pos'][i], dtype=dtype)
        if sys_type=='euler':
            ...
        if sys_type=='lagrange':
            world_pos = torch.as_tensor(data['world_pos'][i], dtype=dtype)
            world_pos_prev = torch.as_tensor(data['world_pos'][i-1], dtype=dtype)
            velocity = world_pos - world_pos_prev

        # node attribute and label
        node_type = torch.as_tensor(data['node_type'][i], dtype=dtype)
        x = velocity
        if sys_type=='euler':
            ...
        if sys_type=='lagrange':
            y = torch.as_tensor(data['world_pos'][i+1], dtype=dtype)

        # add noise
        ...

        # calculate the update quantity (if need)
        if label_update:
            if sys_type=='euler':
                ...
            if sys_type=='lagrange':
                y = y - 2*world_pos + world_pos_prev
            x = torch.cat([node_type, x], 1)

        # transform to pygdata
        ...
```

```
return pyg_data
```

搭建网络

这一流程与在圆柱绕流仿真应用中一致, 不再详细介绍.

训练网络

这一流程与在圆柱绕流仿真应用中一致, 不再详细介绍.

预测和评价

推理脚本中的相关部分:

```
def main():
    ...
    # evaluator
    predictor = gnn4s.infer.PredictorLagrange(model, config.device)
    evaluator = gnn4s.evaluation.Evaluator(data_loader, predictor)

    # evaluate
    error, pred = evaluator.evaluate()
    for i in range(len(error)):
        logger.log_info(error[i])
    ...
```

与在圆柱绕流仿真应用中不同, 预测器采用Lagrange类型, 利用网络预测得到的加速度和当前时刻及上一时刻的世界坐标, 进行两次积分, 预测下一时刻的世界坐标. 计算方法为 $q_i^{t+1} = p_i + 2q_i^t - q_i^{t-1}$, 其中 p_i 为 i 节点预测加速度, q_i^t 为 t 时刻的世界坐标.

```
class PredictorLagrange():
    """ Predictor of lagrange system """
    def __init__(self, model, device):
        """ initialization
        Args:
            model: model
            device: computing device
        """
        self.model = model
        self.device = device

        self.model.eval()

    def __call__(self, data):
        """ predicting the solution from the initial time step to the final time
        step
        Args:
            data: data
        """
        predict = []

        pos_prev = data[0].world_pos_prev
        pos_cur = data[0].world_pos
        for step in range(len(data)):
            # copy graph
```

```
g = copy.deepcopy(data[step])
if step>0:
    g.world_pos = pos_next
    g.x[:,1:] = pos_next - pos_cur
    pos_prev = pos_cur
    pos_cur = pos_next
g = g.to(self.device)

# forward propagation
acceleration = self.model(g)
pos_next = (2*pos_cur - pos_prev).to(self.device) + acceleration

# add mask
mask = torch.logical_not(g.mask)
pos_next[mask] = (pos_cur.to(self.device))[mask]

# store
pos_next = pos_next.cpu().detach()
predict.append(pos_next)

return predict
```

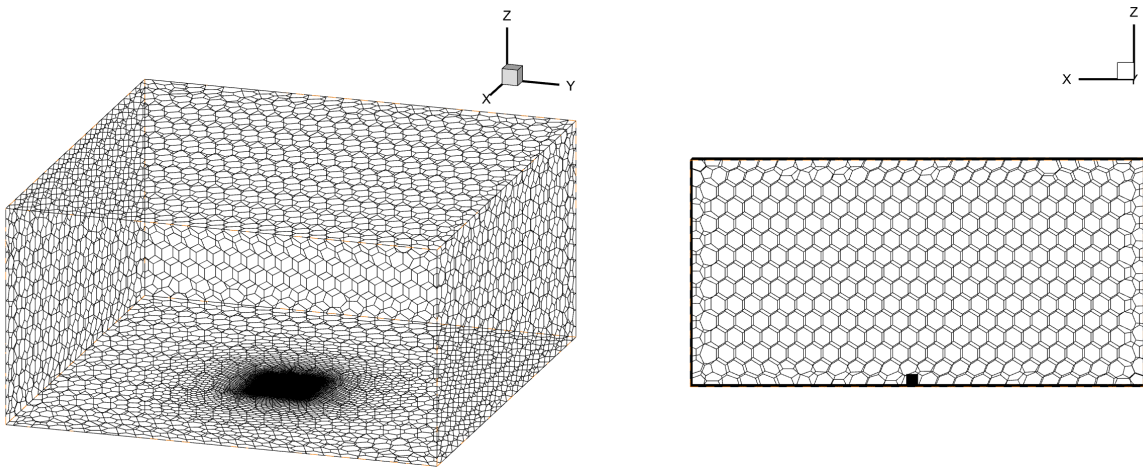
后处理可视化

机翼仿真

这一章我们将应用 MeshGraphNet 模拟机翼附近的流场. 在这个应用中, 训练所用的数据图规模较大. 为减少训练模型所用的内存开销, 我们将原先规模较大的数据图适当分割为一组子图, 以便于后续的处理.

数据集

原始数据文件以 Tecplot 360 EX plot file (.plt) 格式存储, 例如 NACA4412-0deg1.3mach.plt 是迎角为 0° , 马赫数为1.3时流场的参考数据. 用 Tecplot 360 EX 打开, 查看网格.

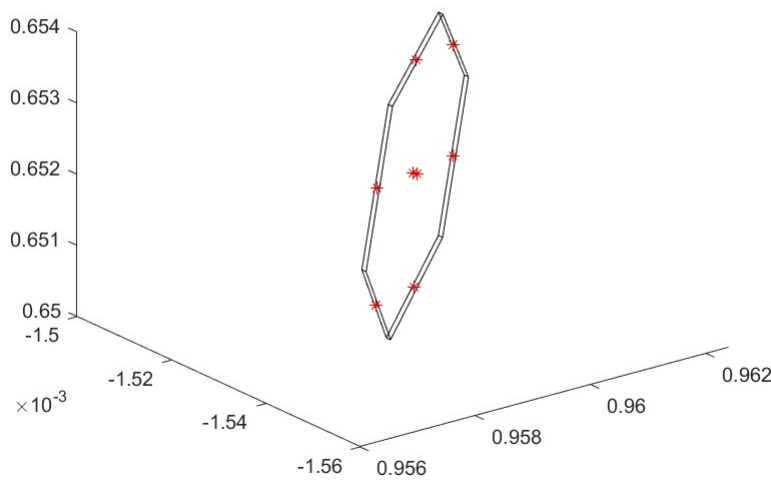


网格信息.

可以看到机翼所在的区域只占整体区域的一小部分. 不过, 机翼附近区域的网格比较密集, 实际上占整体网格的大部分. 整个网格含有数百万个单元.

在 Tecplot 360 EX plot file (.plt) 中使用write data, 将网格及物理属性保存为 .dat 格式文件. .dat 文件含有以下信息:

- 单元顶点的三维坐标;
- 单元中心密度、压力系数和温度速度分量的标签;
- 单元面的顶点索引;
- 面两侧的单元索引.



单元示例

每个单元含有若干个面, 每个面含有若干个顶点. 上图展示了网格中的一个单元, 这个单元含有8个面, 其中2个面含有6个顶点, 另外6个面含有2个顶点.

在原始数据集中, 坐标信息在单元顶点上, 但物理属性却在单元中心上. 虽然单元中心的坐标是未知的, 但是可以利用单元顶点计算得到. 具体地, 首先计算所有面顶点坐标的平均值, 作为面中心的坐标, 然后计算所有面中心坐标的平均值, 作为单元中心的坐标. 尽管这种计算方式有误差, 但误差远小于单元的半径, 并不会成为后续计算过程中的主要误差来源.

统计网格中的单元, 面和顶点的数目, 以 `NACA4412-0deg1.3mach.plt` 为例, 有7,103,508个单元, 42,162,152个面, 32,165,187个顶点.

我们将单元中心的坐标和物理属性加入训练集, 分别作为数据图中顶点的特征和标签. 此外将连接单元中心的边(即面两侧的单元索引)加入数据集中, 作为数据图中的边. 图4展示了部分单元和边的拓扑关系.

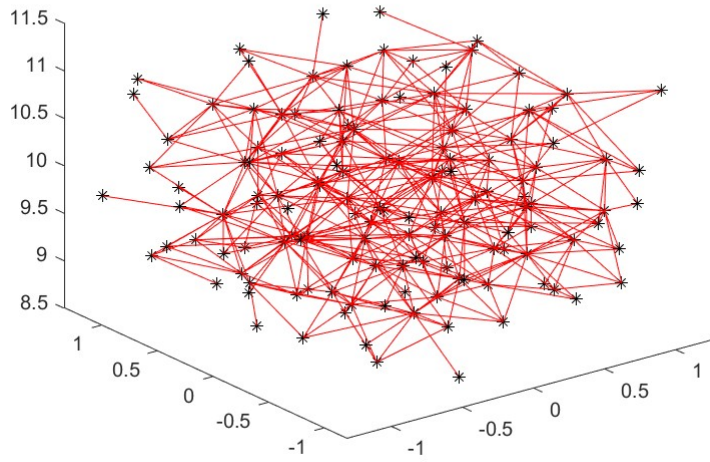


图4 部分单元(黑色)和边(红色)的拓扑关系.

尽管我们不需要考虑网格中的顶点, 只需要考虑单元中心和边, 数据图的规模仍然很大. 估计处理一张图就需要500G-1000G的显存. 为节省内存占用, 我们尝试将数据图分割为一组小规模子图.

具体地, 将整个计算区域 $\Omega := [-20, 20] \times [-20, 20] \times [0, 20]$ 划分为 $n_1 \times n_2 \times n_3$ 块, 每一块为 $\Omega_i = [(x_1^l)^i, (x_1^u)^i] \times [(x_2^l)^i, (x_2^u)^i] \times [(x_3^l)^i, (x_3^u)^i], i = 1, 2, \dots, n_1 \times n_2 \times n_3$. 计算每条边的中心, 选取中心落在 Ω_i 内的边及其连接的单元中心, 作为第 i 张子图.

数据集文件格式: `deg_{deg}_mach_{mach}_part_{part}.h5`, 其中 `deg` 和 `mach` 分别是迎角和马赫数, `part` 是子图的序号. 迎角的范围为 0° - 30° , 马赫数的范围为 1.3-1.7. 每张图分解为约200张子图, 每张子图含有数万个节点.

数据集含有的信息(以字典和numpy数组形式存储):

- 迎角, 马赫数;
- 单元中心的三维坐标;
- 单元中心密度、压力系数和温度速度分量的标签;
- 边两端的单元索引.

读取数据之后, 在前处理阶段为每个顶点和边制作特征:

- 顶点特征记为 v_i , 包含单元中心的三维坐标, 迎角和马赫数;
- 边特征记为 e_{ij} , 包含三维向量 $\overline{v_i v_j}$, 以及向量的长度.

代码

位于 `./demo/foil` 中, 包含训练脚本 `foil_train.py` 和推理脚本 `foil_infer.py`.

文件结构

```
|--dataset                                # 数据集文件
  |--foil
    |--raw                                # 原始数据
      |--deg_0_mach_1.3                  # 训练所用的数据
      ...
      |--deg_30_mach_1.7
    |--foil_train_name_list.txt          # 训练所用的数据清单
    |--foil_valid_name_list.txt          # 验证所用的数据清单
    |--foil_test_name_list.txt           # 推理所用的数据清单
|--checkpoint                            # 检查点
  |--foil
    |--train                             # 训练保存的模型和相关参数设置
    |--infer                             # 推理所用的模型和相关参数设置
    |--foil.pth
|--gnn4s                                 # gnn4s库
  |--data                                # 数据类
    |--dataset.py
    |--load.py
    |--process.py
    |--normalizaiton.py
  |--model                              # 网络模型
    |--gnn.py
    |--mlp.py
  |--train                              # 训练
    |--loss.py
    |--trainer.py
  |--infer                              # 推理
    |--inferencer.py
  |--postprocess                         # 后处理可视化
    |--visualize.py
  |--__init__.py
  |--parser.py                          # 参数
|--config                               # 参数配置文件
  |--foil_train.yaml                    # 训练参数
  |--foil_infer.yaml                    # 推理参数
|--foil_train.py                         # 训练脚本
|--foil_infer.py                         # 推理脚本
```

主要流程

训练 在训练脚本 `foil_train.py` 中, 首先读取参数设置, 并进行一些设置, 这一流程与在之前的应用中一致.

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/foil_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器.

```

# dataset
var_dict = {'node_pos': ['node_pos_x', 'node_pos_y', 'node_pos_z'],
            'node_attr': ['deg', 'mach'],
            'node_label': ['den', 'pre', 'tem', 'vel_x', 'vel_y', 'vel_z']}
load_fn = gnn4s.data.load_h5
process_fn = gnn4s.utils.partial(gnn4s.data.to_pyg_data, var_dict=var_dict)
normalizer = gnn4s.data.Normalizer(var_dict)
train_dataset = gnn4s.data.Dataset(config.data.dir,
f'{config.data.name}_train',
                                load_fn, process_fn, normalizer)
valid_dataset = gnn4s.data.Dataset(config.data.dir,
f'{config.data.name}_valid',
                                load_fn, process_fn, normalizer)

# dataloader
train_dataloader = gnn4s.data.DataLoader(train_dataset)
valid_dataloader = gnn4s.data.DataLoader(valid_dataset, shuffle=False)

```

接下来搭建模型.

```

# model
model = gnn4s.model.GNN(var_dict, config.model.node_input_dim,
                        config.model.edge_input_dim, config.model.output_dim,
                        config.model.hidden_dim, config.model.block_num)
model.to(config.device)

```

构建损失函数和误差函数.

```

# loss and error
loss = gnn4s.train.LossLp(model=model, var_dict=var_dict)
error = gnn4s.train.ErrorRelativeLp(model=model, var_dict=var_dict)

```

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```

# train
optimizer = gnn4s.train.AdamOptimizer(model.parameters(), lr=config.train.lr)
scheduler = gnn4s.train.StepLR(optimizer,
step_size=config.train.scheduler_step,
                                gamma=config.train.scheduler_gamma)

trainer = gnn4s.train.Trainer(

```

```

train_dataloader=train_dataloader, valid_dataloader=valid_dataloader,
model=model, loss=loss, error=error,
optimizer=optimizer, scheduler=scheduler,
epoch_num=config.train.epoch_num, print_interval=10,
save_dirt=config.output.dirt,
ckpt_dirt=config.ckpt.dirt, ckpt_name=config.ckpt.name,
logger=logger, device=config.device)
trainer.train()

```

以上是训练的整个流程.

推理 在推理脚本 `foil_infer.py` 中, 首先读取参数设置和checkpoint.

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/foil_infer.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)

    # load checkpoint
    ckpt_dict =
    gnn4s.utils.load_ckpt(f'{config.ckpt.dirt}/{config.ckpt.name}.pth')
    ckpt_config = ckpt_dict['config']
    logger.log_config(vars(ckpt_config))

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器.

```

# dataset
var_dict = {'node_pos': ['node_pos_x', 'node_pos_y', 'node_pos_z'],
            'node_attr': ['deg', 'mach'],
            'node_label': ['den', 'pre', 'tem', 'vel_x', 'vel_y', 'vel_z']}
load_fn = gnn4s.data.load_h5
process_fn = gnn4s.utils.partial(gnn4s.data.to_pyg_data, var_dict=var_dict)
normalizer = gnn4s.data.Normalizer(var_dict)
normalizer.load_mean_and_std(ckpt_dict['normalizer_state_dict']['mean'])

test_dataset = gnn4s.data.Dataset(config.data.dirt,
f'{config.data.name}_test',
                                load_fn, process_fn, normalizer)
test_dataloader = gnn4s.data.DataLoader(test_dataset)

```

搭建图网络, 并读取模型参数.


```

# model
model = gnn4s.model.GNN(var_dict, ckpt_config.model.node_input_dim,
                        ckpt_config.model.edge_input_dim,
                        ckpt_config.model.output_dim,
                        ckpt_config.model.hidden_dim,
                        ckpt_config.model.block_num)
model.load_state_dict(ckpt_dict['model_state_dict'])
model.to(config.device)

```

构造推理器进行预测.

```

# infer
inferencer = gnn4s.infer.InferencerCat(data_loader=test_data_loader,
                                       normalizer=normalizer, model=model, device=config.device)
pre_dict = inferencer.inference(config.output.dirt)

```

最后进行后处理, 用预测解作图.

```

# visualization
for data_name in pre_dict:
    gnn4s.postprocess.interpolation_data(data_name, config.output.dirt,
                                       './dataset/foil/raw/foil_sec_z_0.09.h5',
                                       f'{config.output.dirt}/{data_name}_pre.h5', var_dict)
    gnn4s.postprocess.plot_foil_anime()

```

以上是推理的整个流程.

参数设置

训练所用的参数设置存放在 `./config/foil_train.yaml` 中.

```

data:
  # name of the dataset
  name: foil
  # directory for storing the dataset
  dirt: ./dataset/foil/raw

model:
  # dimension of node input features
  node_input_dim: 5
  # dimension of elem input features
  elem_input_dim: 5
  # dimension of edge input features
  edge_input_dim: 4
  # dimension of output
  output_dim: 6
  # number of hidden block
  block_num: 10
  # dimension of hidden layer
  hidden_dim: 64

train:
  # learning rate
  lr: 0.0001

```

```
# number of epochs
epoch_num: 1000
# decay step
scheduler_step: 100
# decay rate
scheduler_gamma: 0.98

ckpt:
  # name of checkpoint
  name: foil
  # directory for storing checkpoint
  dirt: ./checkpoint/foil/train

log:
  # directory for saving log
  dirt: ./log/foil

output:
  # name of the dataset
  name: foil
  # directory for storing the dataset
  dirt: ./dataset/foil/output

# computing device
device: cuda
```

推理所用的参数设置存放在 `./config/foil_infer.yaml` 中.

```
data:
  # name of the dataset
  name: foil
  # directory for storing the dataset
  dirt: ./dataset/foil/raw

ckpt:
  # name of checkpoint
  name: foil
  # directory for storing checkpoint
  dirt: ./checkpoint/foil/infer

log:
  # directory for saving log
  dirt: ./log/foil

output:
  # name of the dataset
  name: foil
  # directory for storing the dataset
  dirt: ./dataset/foil/output

# computing device
device: cuda
```

读取数据

训练脚本中的相关部分:

```
...
# dataset
var_dict = {'node_pos': ['node_pos_x', 'node_pos_y', 'node_pos_z'],
            'node_attr': ['deg', 'mach'],
            'node_label': ['den', 'pre', 'tem', 'vel_x', 'vel_y', 'vel_z']}

load_fn = gnn4s.data.load_h5
process_fn = gnn4s.utils.partial(gnn4s.data.to_pyg_data, var_dict=var_dict)
normalizer = gnn4s.data.Normalizer(var_dict)
normalizer.load_mean_and_std(ckpt_dict['normalizer_state_dict']['mean'])

test_dataset = gnn4s.data.Dataset(config.data.dir,
f'{config.data.name}_test',
                                load_fn, process_fn, normalizer)
test_dataloader = gnn4s.data.DataLoader(test_dataset)
...
```

推理脚本中的相关部分类似的.

由于速度的各分量和其它物理量的量级存在差异, 需要进行标准化. `Normalizer` 位于 `gnn4s/dataset/normalization.py` 中, 功能是计算数据的均值和标准差, 并执行标准化和逆标准化.

```
class Normalizer():
    def __init__(self, var_dict):
        """ initialization
        Args:
            var_dict: keys of the variables (postion, attribute and label of the
nodes)
        """
        self.var_dict = var_dict

    def cal_mean_and_std(self, data_list):
        """ calculate mean and std of the node label """
        self.mean = {}
        self.std = {}
        for key in self.var_dict['node_label']:
            self.mean[key] = 0.0
            self.std[key] = 0.0
            node_sum = 0

            for data in data_list:
                self.mean[key] += data.node_label[key].sum(0)
                node_sum += data.node_label[key].shape[0]
            self.mean[key] /= node_sum

            for data in data_list:
                self.std[key] += ((data.node_label[key] -
self.mean[key])**2).sum(0)
            self.std[key] = (self.std[key]/node_sum)**0.5
        ...
    def normalize(self, data):
        """ implement normalization """
```

```

for key in self.var_dict['node_label']:
    data.node_label[key] = (data.node_label[key]-self.mean[key]) /
        (self.std[key]+1e-8)

def inverse_normalize(self, data):
    """ implement inverse normalization """
    for key in self.var_dict['node_label']:
        data[key] = data[key]*self.std[key] + self.mean[key]

```

搭建网络

这一流程与在之前的应用中类似, 不再详细介绍.

训练网络

这一流程与在之前的应用中类似, 不再详细介绍.

预测和评价

推理脚本中的相关部分:

```

def main():
    ...
    inferencer = gnn4s.infer.InferencerCat(dataloader=test_data_loader,
                                           normalizer=normalizer,
                                           model=model, device=config.device)
    pre_dict = inferencer.inference(config.output.dir)
    ...

```

推理器的功能是预测所有子图上的物理量, 并将属于同一张数据图的子图上的预测结果拼接起来.

```

class InferencerCat():
    def __init__(self, data_loader, normalizer, model, device):
        ...
    def inference(self, dir):
        """ implement inference
        Args:
            dir: directory for saving prediction
        """
        pre_dict = {}
        for data in self.data_loader:
            data = data.to(self.device)

            # data name
            data_name = data[0].name.split("/")[0]
            if data_name not in pre_dict:
                pre_dict[data_name] = {}

            # node position
            if 'node_pos_x' not in pre_dict[data_name]:
                pre_dict[data_name]['node_pos_x'] = []
                pre_dict[data_name]['node_pos_y'] = []
                pre_dict[data_name]['node_pos_z'] = []
            pre_dict[data_name]['node_pos_x'].append(
                data['node_pos'][0:1].detach().cpu())

```

```

pre_dict[data_name]['node_pos_y'].append(
    data['node_pos'][:,1:2].detach().cpu())
pre_dict[data_name]['node_pos_z'].append(
    data['node_pos'][:,2:3].detach().cpu())

# predition
pre = self.model(data)
for key in pre:
    pre[key] = pre[key].detach().cpu()
self.normalizer.inverse_normalize(pre)

for key in pre:
    if key not in pre_dict[data_name]:
        pre_dict[data_name][key] = []
    pre_dict[data_name][key].append(pre[key])

# save prediction
for data_name in pre_dict:
    for key in pre_dict[data_name]:
        pre_dict[data_name][key] = torch.cat(pre_dict[data_name]
[key]).numpy()

    os.makedirs(dirt, exist_ok=True)
    with h5py.File(f'{dirt}/{data_name}_pre.h5', "w") as f:
        for key, val in pre_dict[data_name].items():
            f.create_dataset(key, data=val)

return pre_dict

```

后处理可视化

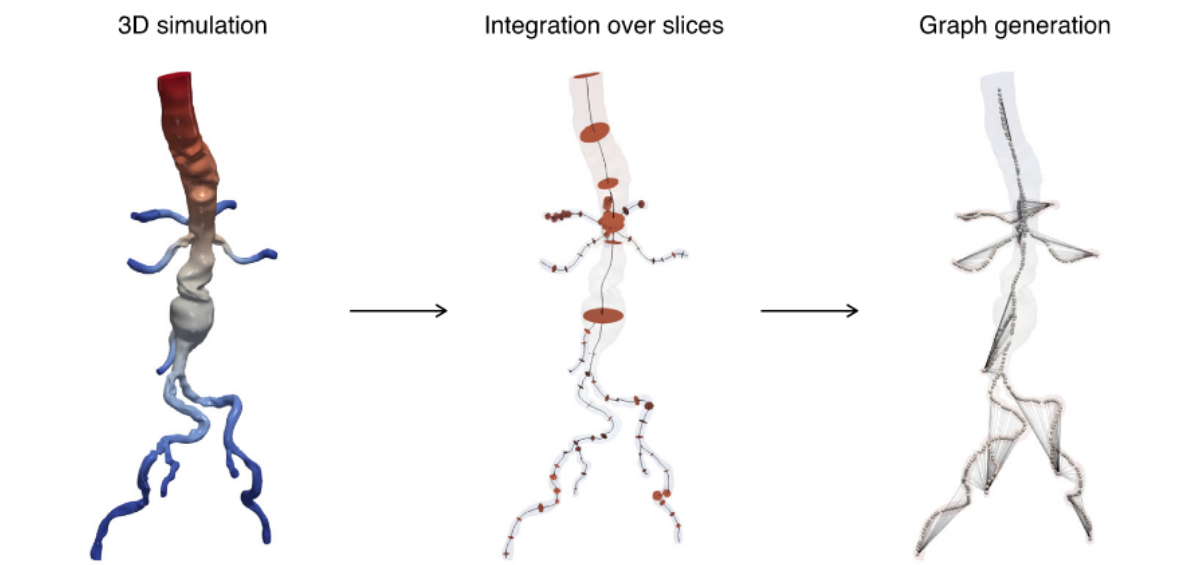
心血管血流模拟

这一章我们将继续介绍如何使用gnn4s实现MeshGraphNet的另一个应用[]: 心血管血流模拟. 与之前介绍的两个应用有所不同, 在这个应用中我们将对原始三维模型进行适当简化, 引入一维模型进行建模; 对图数据进行适当的前处理, 提取更多有用的节点特征和边特征; 并对模型进行适当修改, 支持多步预测.

算法与模型

一维模型

一维模型是通过将三维N-S方程在血管横截面上积分, 以将方程简化为一个空间变量. 具体而言, 动脉树被近似为由代表血管中心线的段组成, 压力, 流速和血管壁位移被视为仅是轴向分量的函数. 与集总参数网络模型相比, 一维模型由于能够考虑流体与弹性血管壁相互作用, 从而能够更全面地捕捉物理现象.



生成MeshGraphNet输入数据的流程. 首先进行三维模拟, 然后在切片上对压力和速度进行积分, 将结果限制在中心线上, 最后从中心线节点和连接生成图形以及相对节点和边的特征 (图片来源于[]).

节点特征和边特征

在这个应用中, 节点和边特征基于相关先验知识提取. 节点特征为17维向量

$$\boldsymbol{v}_i^k = [p_i^k, q_i^k, A_i, \boldsymbol{\alpha}_i^T, \boldsymbol{\Phi}_T, T_{cc}, p_{\min}, p_{\max}, R_{i,p}, C_i, R_{i,d}, l^k]^T \in \mathbb{R}^{17},$$

其中各元素的定义如下.

节点特征	定义
p_i^k	t^k 时刻的压力
q_i^k	t^k 时刻的流速
A_i	横截面积
$\boldsymbol{\alpha}_i^T$	节点类型
$\boldsymbol{\Phi}_T$	中心线切线
T_{cc}	心脏周期持续时间
$p_{\min}$	最小压力 (整个心脏周期)

节点特征	定义
$p_{\max}$	最大压力（整个心脏周期）
$R_{i,p}$	RCR边界条件中的参数 $R_{i,p}$
C_i	RCR边界条件中的参数 C_i
$R_{i,d}$	RCR边界条件中的参数 $R_{i,d}$
l^k	t^k 时刻的布尔负荷变量

边特征为

$$\mathbf{w}_{ij} = [\mathbf{d}_{ij}^T / \|\mathbf{d}_{ij}\|_2, z_{ij}, \boldsymbol{\beta}_{ij}^T]^T \in \mathbb{R}^8,$$

其中各元素的定义如下.

边特征	定义
$\mathbf{d}_{ij}^T / \ \mathbf{d}_{ij}\ _2$	节点 i 和 j 之间的归一化向量距离
z_{ij}	节点 i 和 j 之间的最短路径长度
$\boldsymbol{\beta}_{ij}$	边的类型

多步预测

在这个应用中, 神经网络接受节点特征 $\cup_i \{\mathbf{v}_i\}$ 和边特征 $\cup_{ij} \{\mathbf{w}_{ij}\}$ 作为输入, 输出血管中流速 q 和压力 p 在某些时间段内的变化量 $[\delta q, \delta p]$. 记为 Ψ_m 为应用图网络 m 次后的结果, 即

$$\begin{aligned}\Psi_1(\Theta^k) &= \bigcup_i \{\mathbf{v}_i^k + \delta \mathbf{v}_i^k\} \cup \bigcup_{ij} \{\mathbf{w}_{ij}\}, \\ \Psi_m(\Theta^k) &= \underbrace{(\Psi_1 \circ \dots \circ \Psi_1)}_m(\Theta^k).\end{aligned}$$

我们在网络上进行 s 个连续时间步的训练, 以逼近节点压力和流速的真值 $\hat{p}_i^{k,g}$ 和 $\hat{q}_i^{k,g}$. 这里取 $s = 5$. 定义分步均方误差

$$\text{sMSE}^{k,g,s} = \frac{1}{N_g} \sum_{i=1}^s \sum_{i=1}^{N_g} a_l b_i [(\hat{p}^{k+l,g} - \Psi_l(\Theta^{k,g})|_{p,i})^2 + (\hat{q}^{k+l,g} - \Psi_l(\Theta^{k,g})|_{q,i})^2]^2,$$

其中如果 $l = 1$, 则令 $a_l = 1$, 否则令 $a_l = 0.5$, 同时对于边界节点, 令 $b_i = 100$, 其他情况下令 $b_i = 1$. 整体的损失函数为

$$L = \sum_{g \in \tau} \frac{1}{|\tau|(M^g - s)} \sum_{k=0}^{M^g - s} \text{sMSE}^{k,g,s}.$$

代码

位于 `./demo/vardiovascular` 中, 包含训练脚本 `vardiovascular_train.py` 和推理脚本 `vardiovascular_infer.py`.

文件结构

```
|--dataset                                # 数据集文件
    |--vadiovascular
        |--raw                            # 原始数据
            |--vadiovascular_train.npy    # 训练所用的数据
            |--vadiovascular_valid.npy    # 推理所用的数据
|--checkpoint                            # 检查点
    |--vadiovascular
        |--train                          # 训练保存的模型和相关参数设置
        |--infer                          # 推理所用的模型和相关参数设置
            |--vadiovascular.pth          # 参考模型和相关参数设置
|--gnn4s                                # gnn4s库
    |--data                              # 数据类
        |--dataset.py
        |--load.py
        |--process.py
        |--normalization.py
    |--model                             # 网络模型
        |--mesh_graph_net.py
        |--mlp.py
    |--train                             # 训练
        |--loss.py
        |--trainer.py
    |--evolution                          # 演进过程
        |--predictor.py
    |--evaluation                         # 评价
        |--evaluator.py
    |--postprocess                        # 后处理可视化
        |--visualize.py
    |--__init__.py
    |--parser.py                          # 参数
|--config                                # 参数配置文件
    |--vadiovascular_train.yaml          # 训练参数
    |--vadiovascular_infer.yaml          # 推理参数
|--vadiovascular_train.py                # 训练脚本
|--vadiovascular_infer.py                # 推理脚本
```

主要流程

训练 在训练脚本 `cardiovascular_train.py` 中, 首先读取参数设置, 并进行一些设置, 这一流程与在之前的应用中一致.

```
import torch
import json
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/cardiovascular_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)
```


然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器.

```
# feature
nodes_features = ['area', 'tangent', 'type', 'T', 'dip', 'sysp',
                  'resistance1', 'capacitance', 'resistance2', 'loading']
edges_features = ['rel_position', 'distance', 'type']
features = {'nodes_features': nodes_features, 'edges_features':
edges_features}

# types_to_keep: list of graph types to keep
# 'synthetic' refers to the bcs, not the geometry
dataset_info = json.load(open(f'{config.data.dir}/dataset_info.json'))
types_to_keep = ['synthetic_aorta_coarctation', 'synthetic_pulmonary',
                  'synthetic_aortofemoral']
types_to_keep = {'dataset_info': dataset_info, 'types_to_keep':
types_to_keep}

# dataset
normalize_fn = gnn4s.utils.partial(gnn4s.data.generate_normalized_graphs,
    norm_type=config.data.norm_type, bc_type=config.data.bc_type,
    types_to_keep=types_to_keep, n_graphs_to_keep=-1, features=features)
split_fn = gnn4s.utils.partial(gnn4s.data.split, split_type='train',
    dataset_info=dataset_info)
process_fn = gnn4s.utils.partial(gnn4s.data.graph_to_lightgraph,
    stride=config.model.stride)
noise_fn = gnn4s.utils.partial(gnn4s.data.noise.add_noise_to_lightgraph,
    noise_rate=config.data.noise_rate,
    noise_rate_feat=config.data.noise_rate_feat,
    stride=config.model.stride)
dataset = gnn4s.data.DglDataset(config.data.name, config.data.dir, split_fn,
    load_fn=gnn4s.data.load_graphs, normalize_fn=normalize_fn,
    process_fn=process_fn, noise_fn=noise_fn)

# data loader
data_loader = gnn4s.data.DglDataLoader(dataset,
batch_size=config.data.batch_size)
```

接下来搭建MeshGraphNet网络.

```
# model
model = gnn4s.model.DglMeshGraphNet(config.model.input_dim_node,
    config.model.input_dim_edge, config.model.output_dim,
    config.model.hidden_dim_gnn, config.model.hidden_dim_mlp,
    config.model.layer_num_gnn, config.model.layer_num_mlp)
model.to(config.device)
```

构造均方损失函数.

```
# loss
loss = gnn4s.train.LossMSE(model, config.model.stride, dataset.bc_type)
```

将训练数据, 网络和损失函数传入训练器, 训练网络.

```

# train
optimizer = torch.optim.Adam(model.parameters(), config.train.lr,
                               weight_decay=config.train.weight_decay)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=config.train.epoch_num,
    eta_min=config.train.lr*config.train.lr_decay)

trainer = gnn4s.train.Trainer(data_loader=data_loader, model=model, loss=loss,
                               optimizer=optimizer, lr=config.train.lr, scheduler=scheduler,
                               epoch_num=config.train.epoch_num, logger=logger, device=config.device)
model, history = trainer.train()

```

最后进行后处理, 将训练流程中的损失函数演化情况作图.

```

# postprocess
for k in history:
    gnn4s.postprocess.plot_history(history[k], k, k, config.output.dirt)

```

以上是训练的整个流程.

推理 在推理脚本 `flag_simple_infer.py` 中, 首先读取参数设置. 这一流程在之前的应用中一致.

```

import json
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/cardiovascular_infer.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)

    # load checkpoint
    ckpt_dict =
    gnn4s.utils.load_ckpt(f'{config.ckpt.dirt}/{config.ckpt.name}.pth')
    ckpt_config = ckpt_dict['config']
    logger.log_config(vars(ckpt_config))

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器.

```

# dataset
dataset_info = json.load(open(f'{config.data.dirt}/dataset_info.json'))
types_to_keep = ['synthetic_aorta_coarctation',
                  'synthetic_pulmonary',
                  'synthetic_aortofemoral']
types_to_keep = {'dataset_info': dataset_info, 'types_to_keep':
types_to_keep}

normalize_fn = gnn4s.utils.partial(gnn4s.data.generate_normalized_graphs,
    norm_type=ckpt_config.data.statistics['normalization_type'],
    bc_type=ckpt_config.data.bc_type,
    types_to_keep = types_to_keep,
    statistics=ckpt_config.data.statistics)

```

```

split_fn = gnn4s.utils.partial(gnn4s.data.split, split_type='test',
                                dataset_info=dataset_info)
process_fn = gnn4s.utils.partial(gnn4s.data.graph_to_lightgraph,
                                  stride=ckpt_config.data.stride)
noise_fn = gnn4s.utils.partial(gnn4s.data.noise.add_noise_to_lightgraph,
                                noise_rate=ckpt_config.data.rate_noise,
                                noise_rate_feat=ckpt_config.data.rate_noise_features,
                                stride=ckpt_config.data.stride)

dataset = gnn4s.data.DglDataset(config.data.name, config.data.dirt,
                                 load_fn=gnn4s.data.load_graphs, normalize_fn=normalize_fn,
                                 split_fn=split_fn, process_fn=process_fn, noise_fn=noise_fn)

```

搭建MeshGraphNet网络, 并读取模型参数.

```

# model
model = gnn4s.model.DglMeshGraphNet(ckpt_config.model.input_dim_node,
                                     ckpt_config.model.input_dim_edge, ckpt_config.model.output_dim,
                                     ckpt_config.model.hidden_dim_gnn, ckpt_config.model.hidden_dim_mlp,
                                     ckpt_config.model.layer_num_gnn, ckpt_config.model.layer_num_mlp)
model.load_state_dict(ckpt_dict['model_state_dict'])

```

构造预测器和评价器, 预测解, 并评价预测解的误差.

```

# evaluate
predictor = gnn4s.infer.PredictorCardiovascular(model,
                                                  ckpt_config.data.bc_type)
evaluator = gnn4s.evaluation.EvaluatorCardiovascular(dataset, predictor,
                                                       ckpt_config.data.bc_type, ckpt_config.data.statistics)
pred, error_mean, time_mean, time_step_mean = evaluator.evaluate()
print('average error')
print(error_mean)

```

最后进行后处理, 用预测解作图.

```

# postprocess
idx = 0
gnn4s.postprocess.plot_anime_3d(dataset.data_list[idx].ndata['x'], pred[idx],
                                 dataset.data_list[idx].ndata['nfeatures'],
                                 name='flowrate', dirt=config.output.dirt)

```

以上是推理的整个流程.

参数设置

训练和推理脚本中的相关部分与在之前的应用中一致.

训练所用的参数设置存放在yaml文件 `cardiovascular_train.yaml` 中.

```

data:
  # name of the dataset
  name: cardiovascular
  # directory for storing the dataset
  dirt: ./dataset/cardiovascular/processed

```

```
# rate noise
noise_rate: 100
# rate noise features
noise_rate_feat: 0.00001
# 0: min_max, 1: normal, 2: none
label_norm: 1
# boundary type
bc_type: 'physiological'
# normal type
norm_type: 'normal'
# batch size
batch_size: 100

model:
  # dimension of node input
  input_dim_node: 18
  # dimension of edge input
  input_dim_edge: 9
  # dimension of output
  output_dim: 2
  # latent size gnn
  hidden_dim_gnn: 16
  # latent size mlps
  hidden_dim_mlp: 64
  # hidden layers mlps
  layer_num_mlp: 3
  # gnn layers
  layer_num_gnn: 3
  # stride for multistep training
  stride: 5

train:
  # learning rate
  lr: 0.001
  # learning rate decay
  lr_decay: 0.001
  # l2 regularization
  weight_decay: 0.00001
  # total number of epochs
  epoch_num: 3

output:
  # name of the output file
  name: cardiovascular
  # directory for storing the output file
  dirt: ./dataset/cardiovascular/output

log:
  # directory for saving log
  dirt: ./log/cardiovascular

# computing device
device: cpu
```

推理所用的参数设置存放在 `./config/flag_simple_infer.yaml` 中.

```
data:
  # name of the dataset
  name: cardiovascular
  # directory for storing the dataset
  dirt: ./dataset/cardiovascular/processed

ckpt:
  # name of checkpoint
  name: cardiovascular
  # directory for storing checkpoint
  dirt: ./checkpoint/cardiovascular/infer

output:
  # name of the output file
  name: cardiovascular
  # directory for storing the output file
  dirt: ./dataset/cardiovascular/output
  # output image resolution.
  dpi: 100

log:
  # directory for saving log
  dirt: ./log/cardiovascular

# computing device
device: cuda
```

读取数据

这一流程与在之前两个应用中的不同在于读取和处理数据时需要考虑更多类型的特征. 在这个应用中, 数据图的节点特征 `nodes_features = ['area', 'tangent', 'type', 'T', 'dip', 'sysp', 'resistance1', 'capacitance', 'resistance2', 'loading']`, 边特征包含 `edges_features = ['rel_position', 'distance', 'type']`. 这些列表将会传入数据类中, 按列表读取需要的数据并添加进数据图中.

搭建网络

这一流程与在圆柱绕流仿真应用中类似, 不再详细介绍.

训练网络

这一流程与在圆柱绕流仿真应用中类似, 不再详细介绍.

预测和评价

推理脚本中的相关部分:

```

def main():
    ...
    # evaluator
    predictor = gnn4s.infer.PredictorCardiovascular(model,
ckpt_config.data.bc_type)
    evaluator = gnn4s.evaluation.EvaluatorCardiovascular(dataset, predictor,
        ckpt_config.data.bc_type, ckpt_config.data.statistics)
    pred, error_mean, time_mean, time_step_mean = evaluator.evaluate()
    ...

```

预测器采用Euler类型, 在分支节点处, 需要计算流速的均值.

```

class PredictorCardiovascular():
    """ predictor of lagrange system """
    def __init__(self, model, bc_type):
        """ initialization
        Args:
            model: model
            bc_type: boundary condition type
        """
        ...

    def get_bc_mask(self, graph, time_index):
        """ get boundary conditions mask
        Args:
            graph: DGL graph
            time index: timestep index
        """
        ...

    def perform_timestep(self, graph, bc, time_index, set_bc=True):
        """ performs a single timestep of the rollout phase
        Args:
            graph: DGL graph
            bc: 3D array containing the boundary conditions.
                dim 1: node index, dim 2: pressure (0) and flow rate (1),
                dim 3: timesteps
            time index: index of timestep where we have to take the boundary
                conditions from
            set_bc: set boundary conditions.
        Returns:
            2D array where dim 1 corresponds to node indices,
            and dim 2 corresponds to pressure (0) and flow rate (1)
        """
        gnf = graph.ndata['nfeatures'][:,0:2]
        graph.ndata['next_flowrate'] = bc[:,1,time_index]

        if 'dirichlet' in self.bc_type:
            mask = self.get_bc_mask(graph, time_index)
            gnf[mask] = bc[:,1,time_index][mask]
        elif self.bc_type == 'physiological':
            mask = graph.ndata['inlet_mask'].bool()

        gnf = gnf + self.model(graph)

```

```

        if set_bc:
            if 'dirichlet' in self.bc_type:
                gnf[mask] = bc[:, :, time_index][mask]
            elif self.bc_type == 'physiological':
                gnf[mask, 1] = bc[:, :, time_index][mask, 1]

        return gnf

def compute_average_branches(self, graph, flowrate):
    """ average flowrate over branch nodes
    Args:
        graph: DGL graph
        flowrate: nodal flow rate
    """
    branch_id = graph.ndata['branch_id'].detach().numpy()
    bmax = np.max(branch_id)
    for i in range(bmax + 1):
        idxs = np.where(branch_id==i)[0]
        rflowrate = torch.mean(flowrate[idxs])
        flowrate[idxs] = rflowrate

def __call__(self, graph, average_branches=True):
    """ performs rollout phase
    Args:
        graph: DGL graph
        average_branches: if Trues, averages flowrate over branch nodes.
    Returns:
        2D array of reconstructed features, where dim 1 corresponds to node
        indices and dim 2 corresponds to pressure (0) and flow rate (1),
        2D array containing normalized pressure and flow rate relative errors
        2D array containing pressure and flow rate relative errors
        2D array containing the difference of reconstructed and actual
    features
    """
    times = graph.ndata['nfeatures'].shape[2]
    graph = copy.deepcopy(graph)
    graph_truth = copy.deepcopy(graph)

    gnf_truth = graph_truth.ndata['nfeatures'].clone()
    graph.ndata['nfeatures'] = gnf_truth[:, :, 0].clone()
    graph.edata['efeatures'] =
graph_truth.edata['efeatures'].squeeze().clone()

    gnf = graph.ndata['nfeatures'][:, 0:2].unsqueeze(axis=2).clone()
    for it in range(times-1):
        graph.ndata['nfeatures'][:, -1] = gnf_truth[:, -1, it]
        gf = self.perform_timestep(graph, gnf_truth, it + 1)

        if average_branches:
            self.compute_average_branches(graph, gf[:, 1])

        graph.ndata['nfeatures'][:, 0:2] = gf
        gnf = torch.cat((gnf, gf.unsqueeze(axis = 2)), axis = 2)

    return gnf.detach().numpy()

```

后处理可视化

分子构象生成

模型与算法

分子的构象可以采用图表示, 其中原子和化学键表示为图的节点和边. 记图为 G , 其节点特征和用于表示边的加权邻接矩阵为 X 和 A . 若图 G 具有 N 个节点, 节点特征的维数为 F , 则有 $X \in R^{N \times F}$, $A \in R^{N \times N}$, $G = (X, A) \in R^{N \times F} \times R^{N \times N} := \mathcal{G}$.

扩散模型引入一种前向扩散过程, 将节点特征和邻接矩阵都转换为简单的先验噪声分布, 例如高斯分布. 扩散过程可以表示为在时间范围 $[0, T]$ 内的随机变量轨迹 $\{G_t = (X_t, A_t)\}_{t \in [0, T]}$, 其中 G_0 来自原始数据分布 p_0 . 扩散过程可以由如下的随机微分方程描述:

$$dG_t = f_t(G_t)dt + g_t(G_t)dW, \quad G_0 \sim p_0,$$

其中 $f_t(\cdot) : \mathcal{G} \rightarrow \mathcal{G}$ 是线性漂移系数, $g_t(\cdot) : \mathcal{G} \rightarrow \mathcal{G} \times \mathcal{G}$ 是扩散系数, W 是标准维纳过程, dt 是无穷小时间步长.

扩散过程将原始数据转换为服从简单先验噪声分布的数据. 如果将上述扩散过程逆转, 可以将服从简单先验噪声分布的数据转换为服从原始数据分布的数据. 逆扩散过程也可以由随机微分方程描述:

$$dG_t = [f_t(G_t) - g_t^2(G_t) \nabla_{G_t} \log p_t(G_t)]d\bar{t} + g_t(G_t)d\bar{W},$$

其中 p_t 表示在时间 t 下的前向扩散过程的边际分布, $\bar{W}$ 是逆向时间的标准维纳过程, $d\bar{t}$ 是无穷小时间步长.

GDSS将上述扩散和逆扩散过程分离为关于节点和邻接矩阵的两部分:

$$\begin{cases} dX_t = f_{1,t}(X_t)dt + g_{1,t}dW_1, \\ dA_t = f_{2,t}(A_t)dt + g_{2,t}dW_2, \end{cases}$$
$$\begin{cases} dX_t = [f_{1,t}(X_t) - g_{1,t}^2 \nabla_{X_t} \log p_t(X_t, A_t)]d\bar{t} + g_{1,t}d\bar{W}_1, \\ dA_t = [f_{2,t}(A_t) - g_{2,t}^2 \nabla_{A_t} \log p_t(X_t, A_t)]d\bar{t} + g_{2,t}d\bar{W}_2. \end{cases}$$

选取的先验噪声分布为高斯分布

$$\begin{cases} p_{0t}(X_t|X_0) = \mathcal{N}(X_t; X_0, \sigma_t I), \\ p_{0t}(A_t|A_0) = \mathcal{N}(A_t; A_0, \sigma_t I), \end{cases} \quad \sigma_t = \sigma_{\min}(\sigma_{\max}/\sigma_{\min})^t.$$

根据扩散过程和先验噪声分布, 有

$$f_{1,t} = f_{2,t} = 0, \quad g_{1,t} = g_{2,t} = \frac{d\sigma_t^2}{dt} = 2\sigma_t^2 \sigma_{\min}(\ln \sigma_{\max} - \ln \sigma_{\min}).$$

为将先验噪声转换为服从原始数据分布的数据, GDSS构造两个网络 s_X 和 s_A , 分别学习 $\nabla_{X_t} \log p_t(X_t, A_t)$ 和 $\nabla_{A_t} \log p_t(X_t, A_t)$. 训练两个网络的损失函数为:

$$\begin{cases} L[s_X] := \sum_i (s_X(X_t^i) - \nabla_{X_t^i} \log p_{0t}(X_t^i|X_0))^2 = \sum_i (s_X(X_t^i) + X_t^i/\sigma_t)^2, \\ L[s_A] := \sum_i (s_A(A_t^i) - \nabla_{A_t^i} \log p_{0t}(A_t^i|A_0))^2 = \sum_i (s_A(A_t^i) + A_t^i/\sigma_t)^2. \end{cases}$$

根据逆扩散过程, 我们可以利用训练得到的模型生成服从原始数据分布的数据.

代码

位于 `./demo/mol_gen_qm9` 中, 包含训练脚本 `mol_gen_qm9_train.py` 和推理脚本 `mol_gen_qm9_infer.py`.

文件结构

```
|--dataset                # 数据集文件
  |--qm9
    |--raw                # 原始数据
      |--qm9.csv          # 分子的化学式和化学性质
      |--qm9.npz          # 图数据
      |--valid_idx_qm9.json # 推理所用的数据
|--checkpoint            # 检查点, 训练得到的模型
  |--qm9
|--gnn4s                 # gnn4s库
  |--data                # 数据类
    |--dataset.py
    |--load.py
    |--process.py
  |--model               # 网络模型
    |--gnn_dense.py
    |--mlp.py
  |--diffusion           # 扩散过程
    |--sde.py
    |--predictor_corrector.py
    |--sample.py
  |--train               # 训练
    |--loss.py
    |--trainer.py
  |--evaluation          # 评价
    |--metric.py
  |--postprocess         # 后处理可视化
    |--visualize.py
  |--utils
    |--logger.py         # 记录日志
    |--__init__.py
    |--parser.py         # 参数
|--config                # 参数配置文件
  |--mol_gen_qm9_train.yaml # 训练参数
  |--mol_gen_qm9_infer.yaml # 推理参数
|--mol_gen_qm9_train.py  # 训练脚本
|--mol_gen_qm9_infer.py  # 推理脚本
```

主要流程

训练 在训练脚本 `mol_gen_qm9_train.py` 中, 首先读取参数设置,

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/mol_gen_qm9_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式, 生成数据加载器,

```

# dataset
dataset = gnn4s.data.Dataset(name=config.data.name, dir=config.data.dir,
                              load_fn=gnn4s.data.load_mol, process_fn=gnn4s.data.transform_qm9)
train_idx, test_idx = gnn4s.data.load_idx(
    config.data.dir, config.data.name, dataset.len())
train_dataset, test_dataset = gnn4s.data.split_dataset(
    dataset=dataset, train_idx=train_idx, test_idx=test_idx)

edge_to_adj = gnn4s.data.EdgeToAdj(node_num_max=config.data.node_num_max,
                                     mask=False, channelization=config.data.channelization,
                                     centralization=config.data.centralization,
                                     remove_diagonal=config.data.remove_diagonal)

# data loader
train_data_loader = gnn4s.data.DataLoader(dataset=train_dataset,
                                           batch_size=config.data.batch_size)

```

为图节点和边邻接矩阵分别搭建一个图网络,

```

# model
model_x = gnn4s.model.DenseScoreNetworkX(
    input_dim=config.model.x_input_dim).to(config.device)
model_adj = gnn4s.model.DenseScoreNetworkAdj(
    input_dim=config.model.adj_input_dim).to(config.device)
model = {'model_x': model_x, 'model_adj': model_adj}

```

定义扩散过程,

```

# sde
sde_x = gnn4s.diffusion.VESDE(sigma_min=config.sde.beta_min,
                               sigma_max=config.sde.beta_max,
                               time_steps_num=config.diffusion.time_step_num)

sde_adj = gnn4s.diffusion.VESDE(sigma_min=config.sde.beta_min,
                                 sigma_max=config.sde.beta_max,
                                 time_steps_num=config.diffusion.time_step_num)

```

并根据扩散过程构造损失函数

```

# loss
score_x = gnn4s.train.Score(model=model_x, sde=sde_x, train=True,
continuous=True)
score_adj = gnn4s.train.Score(model=model_adj, sde=sde_adj, train=True,
                              continuous=True)

loss = gnn4s.train.LossDiffusion(sde_x, sde_adj, score_x, score_adj,
edge_to_adj,

                                mask_eps=True, reduce_mean=False)

```

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```

# train
optimizer = gnn4s.train.AdamOptimizer(gnn4s.utils.chain(model_x.parameters(),
model_adj.parameters()),

                                      lr=config.train.lr)
scheduler = gnn4s.train.ExponentialLR(optimizer, gamma=config.train.lr_decay)

trainer = gnn4s.train.Trainer(train_data_loader=train_data_loader, model=model,
                              loss=loss, optimizer=optimizer, lr=config.train.lr, scheduler=scheduler,
                              epoch_num=config.train.epoch_num, logger=logger, device=config.device)
trainer.train()

```

以上是训练的整个流程.

推理 在推理脚本 `mol_gen_qm9_infer.py` 中, 首先读取参数设置, 加载checkpoint,

```

import networkx as nx
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/mol_gen_qm9_infer.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

    # load checkpoint
    ckpt_path = f'{config.model.dir}/{config.model.name}.pth'
    ckpt_dict = torch.load(ckpt_path, map_location=config.device)
    ckpt_config = ckpt_dict['config']
    print(f'Load model checkpoint from {ckpt_path}.')

```

然后读取测试数据, 对训练数据进行一些处理, 转为图数据格式, 并提取其中的测试数据

```

# dataset
dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                             load_fn=gnn4s.data.load_mol,
                             process_fn=gnn4s.data.transform_qm9)
train_idx, test_idx = gnn4s.data.load_idx(
    config.data.dirt, config.data.name, dataset.len())
train_dataset, test_dataset = gnn4s.data.split_dataset(
    dataset=dataset, train_idx=train_idx, test_idx=test_idx)

smiles_data = gnn4s.data.load_smiles(config.data.dirt, config.data.name)
train_smiles =
gnn4s.data.canonicalize_smiles(list(smiles_data.loc[train_idx]))
test_smiles = gnn4s.data.canonicalize_smiles(list(smiles_data.loc[test_idx]))

```

搭建图网络, 并读取测试得到的模型参数,

```

# model
model_x = gnn4s.model.load_model_from_ckpt(
    ckpt_dict['params_x'], ckpt_dict['x_state_dict'], config.device)
model_adj = gnn4s.model.load_model_from_ckpt(
    ckpt_dict['params_adj'], ckpt_dict['adj_state_dict'], config.device)

```

定义扩散过程, 并加载相应的参数设置,

```

# sde
sde_x = gnn4s.diffusion.VESDE(sigma_min=ckpt_config.sde.x.beta_min,
                               sigma_max=ckpt_config.sde.x.beta_max,
                               time_steps_num=ckpt_config.sde.x.num_scales)
sde_adj = gnn4s.diffusion.VESDE(sigma_min=ckpt_config.sde.adj.beta_min,
                                 sigma_max=ckpt_config.sde.adj.beta_max,

time_steps_num=ckpt_config.sde.adj.num_scales)

```

构造采样器, 将模型和扩散过程传入采样器, 用采样器生成一些样本分子,

```

# sampler
sampler = gnn4s.diffusion.PCSampler(model_x=model_x, model_a=model_adj,
                                     sde_x=sde_x, sde_a=sde_adj, continuous=True,
                                     probability_flow=config.sample.probability_flow,
                                     snr=config.sample.signal_noise_ratio, scale_eps=config.sample.scale_eps,
                                     time_steps_num=config.sample.time_steps_num_each,
                                     eps=config.sample.eps, denoise=config.sample.noise_removal,
                                     device=config.device)

# generate samples
train_graph_list = [nx.convert_matrix.from_numpy_array(
    gnn4s.data.edge_idx_to_adj(9,data.edge_index,data.edge_weight))
    for data in train_dataset]
test_graph_list = gnn4s.data.mol_to_nx(gnn4s.data.smiles_to_mol(test_smiles))

shape_x = (inference_sample_size, ckpt_config.data.max_node_num,
           ckpt_config.data.max_feat_num)

```

```

shape_adj = (inference_sample_size, ckpt_config.data.max_node_num,
             ckpt_config.data.max_node_num)

init_mask = gnn4s.data.init_mask(train_graph_list, ckpt_config,
                                 inference_sample_size).to(config.device)
x, adj, _ = sampler.sample(init_mask, shape_x=shape_x, shape_a=shape_adj)
x, adj = gnn4s.data.quantize_qm9(x, adj)
chem_mol = gnn4s.data.ChemMol(x, adj, [6,7,8,9,0])
gen_smiles = gnn4s.data.mol_to_smiles(chem_mol.mol)

```

最后进行后处理, 保存生成的分子并作图, 评价生成分子的质量

```

# save and plot generated samples
gnn4s.data.save_smiles(gen_smiles, output_dirt, f"{config.data.name}-sample")
gnn4s.postprocess.plot_smiles_rdkit(gen_smiles[:16],
    save_path=f'{output_dirt}/{config.data.name}.png')

# evaluation
scores = gnn4s.evaluation.moses_metrics(train_smiles, test_smiles,
                                         gen_smiles, config.device, 8)
scores_nspdk = gnn4s.evaluation.eval_graph_list(test_graph_list,
    gnn4s.data.mol_to_nx(chem_mol.mol), methods=['nspdk'])['nspdk']

logger.log_info(f'number of molecules: {chem_mol.mol_num}')
logger.log_info(f'validity w/o correction:
                {chem_mol.no_correct_num / chem_mol.mol_num}')
for metric in ['valid', f'unique@{len(gen_smiles)}', 'FCD/Test']:
    logger.log_info(f'{metric}: {scores[metric]}')
logger.log_info(f'NSPDK MMD: {scores_nspdk}')

```

以上是推理的整个流程.

参数设置

训练和推理脚本中的相关部分:

```

import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/mol_gen_qm9_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)

```

推理时还需要读取checkpoint

```

# load checkpoint
ckpt_path = f'{config.model.dirt}/{config.model.name}.pth'
ckpt_dict = torch.load(ckpt_path, map_location=config.device)
ckpt_config = ckpt_dict['config']
print(f'Load model checkpoint from {ckpt_path}.')

```

训练所用的参数设置存在 `./config/mol_gen_qm9_train.yaml` 中:

```
data:
  # name of the dataset
  name: qm9
  # directory for storing dataset
  dirt: ./dataset/qm9/raw
  # ratio for splitting dataset
  split_ratio: 0.8
  # max number of node in each graph
  node_num_max: 9
  # whether to rescale data to [-1,1]
  centralization: False
  # whether to add channel
  channelization: False
  # whether to remove diagonal in adjacency matrix
  remove_diagonal: False

  # batch size in training
  batch_size: 1024

model:
  # dimension of input of network x
  x_input_dim: 4
  # dimension of input of network adj
  adj_input_dim: 2
  # number of features
  features_num: 256
  # type of graph layer
  graph_layer_type: PosTransLayer
  # number of graph layer
  graph_layers_num: 4
  # number of channels
  channels_num: 1
  # type of activation function
  actv_type: silu

sde:
  # minimum beta
  beta_min: 0.1
  # maximum beta
  beta_max: 1.0

diffusion:
  # number of time steps in diffusion
  time_step_num: 1000

train:
  # learning rate
  lr: 0.005
  # decay rate
  lr_decay: 0.999
  # number of training epoch
  epoch_num: 100
  # number of warmup step
```

```
warmup_step: 0
# upper bound of norm for gradient clipping
grad_clip: 0
# whether use exponential moving average
use_ema: False

log:
  # directory for saving log
  dirt: ./log/qm9

# device
device: cuda
```

推理所用的参数设置存放在 `./config/mol_gen_qm9_infer.yaml` 中

```
data:
  # name of dataset
  name: qm9
  # directory for storing dataset
  dirt: ./dataset/qm9/raw

ckpt:
  # name of checkpoint
  name: qm9
  # directory for storing checkpoint
  dirt: ./checkpoint/qm9/infer

sample:
  # signal to noise ratio
  signal_noise_ratio: 0.2
  # number of time steps in each stop of correction
  time_steps_num_each: 1
  # scale eps
  scale_eps: 0.7
  # whether or not remove noise
  noise_removal: True
  #
  probability_flow: False
  # eps
  eps: 0.0001

inference:
  # sample size
  sample_size: 100

output:
  # directory for saving output
  dirt: ./dataset/qm9/output

log:
  # directory for saving log
  dirt: ./log/qm9

# device
device: cpu
```


读取数据

训练脚本中的相关部分:

```
def main():
    ...
    # dataset
    dataset = gnn4s.data.Dataset(name=config.data.name, dirt=config.data.dirt,
                                  load_fn=gnn4s.data.load_mol, process_fn=gnn4s.data.transform_qm9)
    train_idx, test_idx = gnn4s.data.load_idx(
        config.data.dirt, config.data.name, dataset.len())
    train_dataset, test_dataset = gnn4s.data.split_dataset(
        dataset=dataset, train_idx=train_idx, test_idx=test_idx)

    edge_to_adj = gnn4s.data.EdgeToAdj(node_num_max=config.data.node_num_max,
                                         mask=False, channelization=config.data.channelization,
                                         centralization=config.data.centralization,
                                         remove_diagonal=config.data.remove_diagonal)
    ...

    # dataloader
    dataloader = gnn4s.data.DataLoader(dataset=train_dataset,
                                         batch_size=config.data.batch_size)
```

推理时还需要读取SMILES字符串, 以用于后续与生成的分子进行比较, 评价分子的质量.

```
smiles_data = gnn4s.data.load_smiles(config.data.dirt, config.data.name)
train_smiles =
gnn4s.data.canonicalize_smiles(list(smiles_data.loc[train_idx]))
test_smiles = gnn4s.data.canonicalize_smiles(list(smiles_data.loc[test_idx]))
```

与在圆柱绕流仿真的应用中相同, 这里仍然可以调用数据类 `Dataset` 和数据加载器 `DataLoader` (位于 `./gnn4s/data/dataset.py` 中), 存放和加载数据. 而传入 `Dataset` 的用于读取和前处理数据的函数 `load_fn` 和 `process_fn` 则与之前有所不同. 这里使用 `gnn4s.data.load_mol` 从数据文件 `qm9.npz` 中读取分子的构象信息, 并使用 `gnn4s.data.transform_qm9` 将读取的分子构象转换为 `PyGData`, 存放在数据类 `Dataset` 中.

创建数据实例 `dataset` 之后, 调用 `gnn4s.data.load_idx` 从 `valid_idx_qm9.json` 中读取测试集的指标, 并调用 `gnn4s.data.split_dataset`, 根据这些指标将数据集划分为 `train_dataset` 和 `test_dataset`, 分别用于训练和测试.

另外, 调用 `gnn4s.data.EdgeToAdj` 创建实例 `edge_to_adj`, 以用于后续在处理图数据时将边的列表转换为邻接矩阵.

搭建网络

训练脚本中的相关部分:

```
def main():
    ...
    # model
    model_x = gnn4s.model.DenseScoreNetworkX(
        input_dim=config.model.x_input_dim).to(config.device)
    model_adj = gnn4s.model.DenseScoreNetworkAdj(
        input_dim=config.model.adj_input_dim).to(config.device)
    model = {'model_x': model_x, 'model_adj': model_adj}
    ...
```

推理时调用 `load_checkpoint()` 读取经过训练的模型.

我们为图中的节点和边分别搭建了一个网络, 用于分别预测扩散过程中与节点和边相关的得分.

```
class DenseScoreNetworkX(nn.Module):
    def __init__(self, input_dim, hidden_dim: int=16, layer_num: int=2,
                  actv=nn.Tanh()) -> None:
        super(DenseScoreNetworkX, self).__init__()
        """ Network for predict the score related to node features.
        Args:
            input_dim: dimension of input
            hidden_dim: dimension of hindden layer
            layer_num: number of hidden layer
            actv: type of activate function
        """
        ...
```

```
class DenseScoreNetworkAdj(nn.Module):
    def __init__(self, input_dim: int=2, nodes_num_max: int=9, attn_input_dim:
int=4,
                  attn_hidden_dim: int=16, attn_layer_num: int=3, attn_heads_num:
int=4,
                  lin_output_dim: int=4, lin_hidden_dim: int=8, lin_layer_num:
int=2,
                  conv_type='GCN') -> None:
        super(DenseScoreNetworkAdj, self).__init__()
        """ Network for predict the score related to node features.
        Args:
            input_dim: dimension of input
            nodes_num_max: maximum number of nodes in each graph
            attn_input_dim: dimension of input of attention layer
            attn_hidden_dim: dimension of hidden layer in attention layer
            attn_layer_num: number of attention layers
            attn_heads_num: number of heads in attention layer
            lin_output_dim: dimension of output of linear layer
            lin_hidden_dim: dimension of hidden layer in linear layer
            lin_layer_num: number of linear layer
            conv_type: type of convolution layer
        """
```

扩散过程

训练脚本中的相关部分:

```
def main():
    ...
    # sde
    sde_x = gnn4s.diffusion.VESDE(sigma_min=config.sde.beta_min,
                                   sigma_max=config.sde.beta_max,
                                   time_steps_num=config.diffusion.time_step_num)
    sde_adj = gnn4s.diffusion.VESDE(sigma_min=config.sde.beta_min,
                                     sigma_max=config.sde.beta_max,
                                     time_steps_num=config.diffusion.time_step_num)
    ...
```

推理时从checkpoint中读取参数设置.

在这个应用中, 用于描述扩散过程的SDE采用 Variance exploding SDE

$$\begin{cases} dX_t = f_{1,t}(X_t)dt + g_{1,t}dW_1, \\ dA_t = f_{2,t}(A_t)dt + g_{2,t}dW_2, \end{cases}$$

其中

$$f_{1,t} = f_{2,t} = 0, \quad g_{1,t} = g_{2,t} = \frac{d\sigma_t^2}{dt} = 2\sigma_t^2\sigma_{\min}(\ln\sigma_{\max} - \ln\sigma_{\min}).$$

噪声边际分布为高斯分布

$$\begin{cases} p_{0t}(X_t|X_0) = \mathcal{N}(X_t; X_0, \sigma_t I), \\ p_{0t}(A_t|A_0) = \mathcal{N}(A_t; A_0, \sigma_t I), \end{cases} \quad \sigma_t = \sigma_{\min}(\sigma_{\max}/\sigma_{\min})^t.$$

调用SDE类 `gnn4s.diffusion.VESDE` 创建实例 `sde_x` 和 `sde_adj`. `gnn4s.diffusion.VESDE` 含有描述扩散过程的函数 `sde`, 计算边际分布的函数 `marginal_prob` 以及从先验分布中生成采样的函数 `prior_sampling` 和 `prior_sampling_sym` (后者将生成对称的样本).

```
class VESDE(SDE):
    def __init__(self, sigma_min: float=0.01, sigma_max: float=50.,
                  time_steps_num: int=1000) -> None:
        """ Variance exploding SDE.
        Args:
            sigma_min: lower bound of standard deviation of the normal
                       distribution of noise
            sigma_max: upper bound of standard deviation of the normal
                       distribution of noise
            time_steps_num: number of discretization steps
        """
        super().__init__(time_steps_num)
        ...

    @property
    def time_end(self):
        """ End time. """
        return 1.0
```

```

def sde(self, x: torch.Tensor, t: torch.Tensor):
    """ Stochastic differential equation.
    Args:
        x: node features or adjacency matrices
        t: time step (between 0 to self.time_end)
    Returns:
        drift: linear drift coefficient
        diffusion: diffusion coefficient
    """
    sigma = self.sigma_min * (self.sigma_max / self.sigma_min) ** t
    drift = torch.zeros_like(x)
    diffusion = sigma * torch.sqrt(torch.tensor(
        2 * (np.log(self.sigma_max) - np.log(self.sigma_min)),
        device=t.device))
    return drift, diffusion

def marginal_prob(self, x, t):
    """ probability density function of the marginal distribution.
    Args:
        x: node features or adjacency matrices
        t: time step (between 0 to self.time_end)
    Returns:
        mean: mean of the probability density function
        std: std of the probability density function
    """
    mean = x
    std = self.sigma_min * (self.sigma_max / self.sigma_min) ** t
    for i in range(len(std.shape), len(mean.shape)):
        std = std.unsqueeze(-1)
    return mean, std

def prior_sampling(self, shape):
    """ Generate samples from a prior distribution.
    Args:
        shape: shape of the samples
    Returns:
        samples
    """
    return torch.randn(*shape)

def prior_sampling_sym(self, shape):
    """ Generate symmetric samples from a prior distribution.
    Args:
        shape: shape of the samples
    Returns:
        samples
    """
    x = torch.randn(*shape).triu(1)
    x = x + x.transpose(-1, -2)
    return x

```

损失函数

训练脚本中的相关部分:

```
def main():
    ...
    # loss
    score_x = gnn4s.train.Score(model=model_x, sde=sde_x, train=True,
                                continuous=True)
    score_adj = gnn4s.train.Score(model=model_adj, sde=sde_adj, train=True,
                                   continuous=True)

    loss = gnn4s.train.LossDiffusion(sde_x, sde_adj, score_x, score_adj,
                                     edge_to_adj,
                                     mask_eps=True, reduce_mean=False)
    ...
```

```
class Score():
    def __init__(self, model, sde, train: bool=False,
                 continuous: bool=False) -> None:
        """ Fractional function that the diffusion model aim to learn.
        Args:
            model: diffusion model
            sde: sde describing the evolution of the diffusion process
            train: whether to train or evaluate the mode
            continue: whether to use a continuous or discrete computation
                    approach
        """
        ...

    if isinstance(self.sde, gnn4s.diffusion.VESDE):
        if continuous:
            def score_fn(x, adj, mask, t):
                ...
                return score
        else:
            def score_fn(x, adj, mask, t):
                ...
                return score

    self.score_fn = score_fn

    def __call__(self, x, adj, mask, t):
        """ Calulate the value that the diffusion model aim to learn.
        Args:
            x: node features
            adj: adjacency matrices
            mask: mask
            t: time
        """
        return self.score_fn(x, adj, mask, t)
```

```

class LossDiffusion():
    def __init__(self, sde_x=None, sde_adj=None, score_x=None, score_adj=None,
                  edge_to_adj=None, symmetric: bool=True, mask_eps: bool=False,
                  reduce_mean: bool=True, likelihood_weighting: bool=False,
                  eps: float=1e-5) -> None:
        """ Loss function for training diffusion model.
        Args:
            sde_x: sde describing the evolution of node features
            sde_adj: sde describing the evolution of adjacency matrices
            score_x: target score function related to node features
            score_adj: target score function related to adjacency matrices
            edge_to_adj: function convert edge index list to adjacency matrices
            symmetric: whether to keep adjacency matrix symmetric
            mask_eps: whether to add a mask for features with small value
            reduce_mean: whether to reduce residuals by taking the mean
            likelihood_weighting: whether to multiply the residuals by weights
            eps: tolerance value
        """
        ...

    def loss_term(self, sde, z, mask, std, score, t):
        """ Loss term related to node or adjacency matrix.
        Args:
            sde: stochastic differential equation describing the diffusion
process
            z: node or adjacency matrix with noise
            mask: mask
            std: standard deviation of the noise distribution
            score: target score function
            t: time
        """
        ...
        if not self.likelihood_weighting:
            loss = torch.square(score * std + z)
            ...
            loss = loss.mean()
        else:
            g2 = sde.sde(torch.zeros_like(z), t)[1] ** 2
            loss = torch.square(score + z / std)
            ...
            loss = (loss*g2).mean()
        return loss

    def __call__(self, batch_data):
        """ Compute the value of loss function
        Args:
            batch_data: a mini-batch of training data, including node features,
adjacency matrices and mask.
        Returns:
            loss: value of loss function
        """
        x, adj, mask_x, mask_adj = self.edge_to_adj(batch_data)

```

```

t = torch.rand(adj.shape[0], device=adj.device) *
    (self.sde_adj.time_end - self.eps) + self.eps
...
if x is not None:
    z_x = torch.randn_like(x)
    ...
    mean_x, std_x = self.sde_x.marginal_prob(x, t)
    perturbed_x = mean_x + std_x * z_x
    ...
if adj is not None:
    z_adj = torch.randn_like(adj)
    ...
    mean_adj, std_adj = self.sde_adj.marginal_prob(adj, t)
    ...
    perturbed_adj = mean_adj + std_adj * z_adj
    ...

if x is not None:
    score_x = self.score_x(perturbed_x, perturbed_adj, mask_x, t)
    loss_x = self.loss_term(self.sde_x, z_x, mask_x, std_x, score_x, t)

if adj is not None:
    score_adj = self.score_adj(perturbed_x, perturbed_adj, mask_adj, t)
    ...
    loss_adj = self.loss_term(self.sde_adj, z_adj, mask_adj, std_adj,
                              score_adj, t)

loss = {}
if loss_x is not None:
    loss['loss_x'] = loss_x
if loss_adj is not None:
    loss['loss_adj'] = loss_adj

return loss

```

训练网络

训练脚本中的相关部分:

```

def main():
    ...
    # train
    optimizer = gnn4s.train.AdamOptimizer(gnn4s.utils.chain(model_x.parameters(),
model_adj.parameters()),
                                          lr=config.train.lr)
    scheduler = gnn4s.train.ExponentialLR(optimizer, gamma=config.train.lr_decay)

    trainer = gnn4s.train.Trainer(train_dataloader=train_dataloader, model=model,
    loss=loss, optimizer=optimizer, lr=config.train.lr, scheduler=scheduler,
    epoch_num=config.train.epoch_num, logger=logger, device=config.device)
    trainer.train()

```

训练时两个网络一同进行训练.

预测

推理脚本中的相关部分:

```
def main():
    ...
    # sampler
    sampler = gnn4s.diffusion.PCSampler(model_x=model_x, model_a=model_adj,
        sde_x=sde_x, sde_a=sde_adj, continuous=True,
        probability_flow=config.sample.probability_flow,
        snr=config.sample.signal_noise_ratio, scale_eps=config.sample.scale_eps,
        time_steps_num=config.sample.time_steps_num_each,
        eps=config.sample.eps, denoise=config.sample.noise_removal,
        device=config.device)

    # generate samples
    train_graph_list = [nx.convert_matrix.from_numpy_array(
        gnn4s.data.edge_idx_to_adj(9,data.edge_index,data.edge_weight))
        for data in train_dataset]
    test_graph_list = gnn4s.data.mol_to_nx(gnn4s.data.smiles_to_mol(test_smiles))

    shape_x = (inference_sample_size, ckpt_config.data.max_node_num,
        ckpt_config.data.max_feat_num)
    shape_adj = (inference_sample_size, ckpt_config.data.max_node_num,
        ckpt_config.data.max_node_num)

    init_mask = gnn4s.data.init_mask(train_graph_list, ckpt_config,
        inference_sample_size).to(config.device)
    x, adj, _ = sampler.sample(init_mask, shape_x=shape_x, shape_a=shape_adj)
    x, adj = gnn4s.data.quantize_qm9(x, adj)
    chem_mol = gnn4s.data.ChemMol(x, adj, [6,7,8,9,0])
    gen_smiles = gnn4s.data.mol_to_smiles(chem_mol.mol)
    ...
```

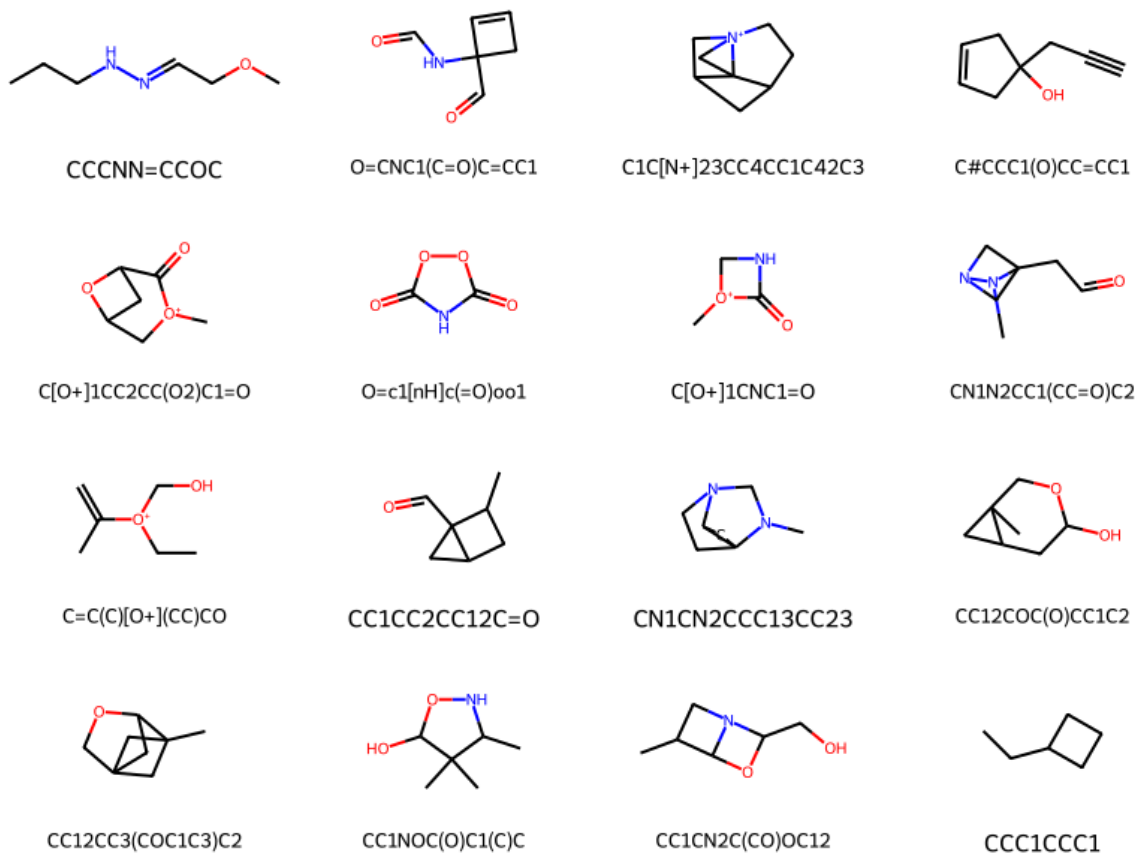
后处理可视化

推理脚本中的相关部分:

```
def main():
    ...
    # save and plot generated samples
    gnn4s.data.save_smiles(gen_smiles, output_dirt, f"{config.data.name}-sample")
    gnn4s.postprocess.plot_smiles_rdkit(gen_smiles[:16],
        save_path=f'{output_dirt}/{config.data.name}.png')
    ...
```

生成分子的SMILES字符串保存在路径 `./data/qm9/output/qm9-sample.txt` 中. 选取部分分子作图, 图像保存在路径 `./data/qm9/output/qm9.png` 中.

部分生成分子的构象:



评价

推理脚本中的相关部分:

```
def main():
    ...
    # evaluation
    scores = gnn4s.evaluation.moses_metrics(train_smiles, test_smiles,
                                           gen_smiles, config.device, 8)
    scores_nspdk = gnn4s.evaluation.eval_graph_list(test_graph_list,
                                                    gnn4s.data.mol_to_nx(chem_mol.mol), methods=['nspdk'])['nspdk']

    logger.log_info(f'number of molecules: {chem_mol.mol_num}')
    logger.log_info(f'validity w/o correction:
                    {chem_mol.no_correct_num / chem_mol.mol_num}')
    for metric in ['valid', f'unique@{len(gen_smiles)}', 'FCD/Test']:
        logger.log_info(f'{metric}: {scores[metric]}')
    logger.log_info(f'NSPDK MMD: {scores_nspdk}')
```

评价结果(每次评价的结果将略有不同):

```
number of molecules: 100
validity w/o correction: 0.94
valid: 1.0
unique@100: 1.0
FCD/Test: 5.396886533085823
NSPDK MMD: 0.012334524111661416
```

分子对接

模型与算法

算法原理

预测小分子配体与蛋白质受体之间的结合构型, 即分子对接, 对于药物设计至关重要. 在这一章节中, 我们将介绍如何使用gnn4构造扩散生成模型(DGM) DIFFDOCK[], 在给定的目标蛋白质受体和配体结构下, 学习一个配体姿态的分布.

DIFFDOCK[] 将分子对接视为一种生成建模问题, 采用一个DGM将一个噪声先验分布逐步转化为学习模型分布. 这个过程可以看作是通过平移, 旋转和扭曲角度的更新逐步优化配体姿态. 此外, DIFFDOCK还训练了一个置信度模型, 用于提供从DGM中采样的姿态的置信度估计, 并选择最可能的样本.

配体构象变换

DIFFDOCK将配体种子构象 c 所属的姿态空间看作是一个 $(m+6)$ 维的子流形 $\mathcal{M}_c \in \mathbb{R}^{3n}$, 其中 n 是原子数量, m 是可旋转键的数量, 而额外的六个自由度来自于相对于固定蛋白质的旋转平移. 分子对接被形式化为在蛋白质结构 y 的条件下, 学习一个在流形 $\mathcal{M}_c$ 上的概率分布 $p_c(x|y)$. DIFFDOCK将 $\mathcal{M}$ 中的元素映射到对应于这些变换的群的乘积空间中, 在这个乘积空间上采样扩散核函数, 并在其中训练一个 DGM.

在拓扑学中, "流形"(manifold)描述了具有局部欧几里德空间特性的空间. 简单来说, 流形是可以通过欧几里德空间的片段进行逼近的空间. 它可以被视为是局部上与欧几里德空间同胚(同构)的空间. 流形可以是多维的, 并且可以是连续的, 光滑的或具有其他特定的性质. 例如, 三维空间中的曲面, 高维空间中的超平面.

在机器学习和数据分析领域, 流形学习是一种方法, 用于在高维数据中寻找低维流形结构, 从而实现数据降维和可视化.

DIFFDOCK将配体位置的平移与三维平移群 $\mathbb{T}(3)$ 关联起来, 将配体的刚性旋转与三维旋转群 $SO(3)$ 关联起来, 将每个可旋转键的扭转角变化与一个复制的二维旋转群 $SO(2)$ 关联起来. 更正式地说, DIFFDOCK 对配体构象 $c \in \mathbb{R}^{3n}$ 上的每个群定义操作.

三维旋转群 $SO(3)$ 代表三维欧几里德空间中的所有旋转操作构成的群. "SO" 代表特殊正交群 (Special Orthogonal Group), 表示旋转矩阵的行列式为正, 并且矩阵的转置和逆矩阵相等. 数字 "3" 表示三维空间.

$SO(3)$ 中的旋转操作是围绕原点进行的, 可以通过旋转角度和旋转轴来描述. 旋转矩阵是一个 3×3 的正交矩阵, 它能够保持向量的长度不变, 并且保持向量之间的夹角不变.

平移操作 $A_{tr} : \mathbb{T}(3) \times \mathbb{R}^{3n} \rightarrow \mathbb{R}^{3n}$ 的定义直接为 $A_{tr}(r, x)_i = x_i + r$, 其中 $x_i \in \mathbb{R}^3$ 是第 i 个原子的位置, 使用同构 $\mathbb{T}(3) = \mathbb{R}^3$.

旋转操作 $A_{rot} : SO(3) \times \mathbb{R}^{3n} \rightarrow \mathbb{R}^{3n}$ 的定义为 $A_{rot}(R, x)_i = R(x_i - \bar{x}) + \bar{x}$, 其中 $\bar{x} = 1/n \sum x_i$, 表示绕配体的质心进行旋转.

扭转操作 $A_{tor} : SO(2)^m \times \mathbb{R}^{3n} \rightarrow \mathbb{R}^{3n}$ 的定义为

$$A_{tor}(\theta, x) = \text{RMSDAlign}(x, B_{1,\theta_1} \circ \dots \circ B_{m,\theta_m}(x))$$

其中 $\theta = (\theta_1, \dots, \theta_m)$, θ_k 是第 k 个可扭转键 (a_k, b_k) 的扭转角度, a_i 和 b_i 是化学键的两侧, $B_{k,\theta_k}(x) \in \mathbb{R}^{3n}$ 是围绕第 k 个可旋转键进行 θ_k 扭转的任何有效扭转更新,

$$\text{RMSDAlign}(x, x') = \arg \min_{x^\dagger \in \{gx' | g \in SE(3)\}} \text{RMSD}(x, x^\dagger),$$

表示按任意顺序应用所有的 m 个扭转更新, 然后对未修改的构象进行全局 RMSD 对齐.

综合上述变换操作, 对应于配体结合自由度的乘积空间为 $\mathbb{P} = \mathbb{T}^3 \times SO(3) \times SO(2)^m$, 配体构象变换 $A: \mathbb{P} \times \mathbb{R}^{3n} \rightarrow \mathbb{R}^{3n}$ 为

$$A((r, R, \theta), x) = A_{tr}(r, A_{rot}(R, A_{tor}(\theta, x))).$$

从而对于一个种子配体构象 c , 可以形式上定义配体构象空间 $\mathcal{M}_c = \{A(g, c) | g \in \mathbb{P}\}$. 这对应于通过刚体运动加上扭转角度灵活性可以达到的配体构象空间的直观概念.

乘积空间上的扩散

DiffDock利用乘积空间 $\mathbb{P}$ 学习一个 DGM 来生成配体构象, 在 $\mathbb{P}$ 上定义了一个扩散过程. 由于 $\mathbb{P}$ 是一个乘积流形, 前向扩散在每个流形中独立进行, 切空间是一个直和:

$$T_g\mathbb{P} = T_r\mathbb{T}^3 \oplus T_RSO(3) \oplus T_\theta SO(2)^m \cong \mathbb{R}^3 \oplus \mathbb{R}^3 \oplus \mathbb{R}^m,$$

其中 $g = (r, R, \theta)$. 因此只需在每个群体中独立地从扩散核中进行采样并回归其得分.

"切空间"(tangent space)是描述流形上某一点的切平面的概念. 切空间可以理解流形上某一点附近的局部线性近似. 它由通过该点的切向量(tangent vector)构成, 这些切向量描述了在该点处的微小位移方向. 切向量是表示切空间中的向量, 可以视为切线或曲面的切线.

在这三个群体中, 正向 SDE 定义为

$$dx = \sqrt{d\sigma^2(t)/dt} dW,$$

其中 $\sigma^2 = \sigma_{tr}^2, \sigma_{rot}^2$ 或 σ_{tor}^2 , 分别对应于 $\mathbb{T}(3)$, $SO(3)$ 和 $SO(2)^m$, 而 W 是相应的布朗运动. 由于 $\mathbb{T}(3) \cong \mathbb{R}^3$, 平移情况是微不足道的, 涉及到对具有方差 $\sigma^2(t)$ 的标准高斯分布进行采样和计算得分. $SO(3)$ 上的扩散核由 $IGSO(3)$ 分布给出, 可以通过在轴角参数化下均匀采样一个单位向量 $\hat{\omega} \in so(3)$ 和按照如下分布:

$$p(\omega) = \frac{1 - \cos \omega}{\pi} f(\omega) \quad \text{where} \quad f(\omega) = \sum_{l=0}^{\infty} (2l+1) \exp(-l(l+1)\sigma^2/2) \frac{\sin((l+1/2)\omega)}{\sin(\omega/2)}$$

采样一个随机角度 $\omega \in [0, \pi]$ 来进行采样. 此外, 扩散核的得分为

$\nabla \ln p_t(R'|R) = (\frac{d}{d\omega} \log f(\omega))\hat{\omega} \in T_{R'}SO(3)$, 其中 $R' = R(\omega\hat{\omega})R$ 是将欧拉向量 $\omega\hat{\omega}$ 应用于 R 的结果. 得分计算和采样可以通过预计算截断无限级数和插值 $p(\omega)$ 的累积分布函数来高效完成. 最后, $SO(2)^m$ 群体与环面 $\mathbb{T}^m$ 同胚, 在该群体上, 扩散核是一个带有方差 $\sigma^2(t)$ 的环绕正态分布. 可以直接进行采样, 并且得分可以作为截断无限级数进行预计算.

模型架构

DiffDock构建了得分模型 $s(x, y, t)$ 和置信度模型 $d(x, y)$, 它们的输入是当前的配体姿态 x 和蛋白质结构 y 在三维空间中的表示. 得分模型的输出位于切空间 $T_r\mathbb{T}^3 \oplus T_RSO(3) \oplus T_\theta SO(2)^m$ 中. 空间 $T_r\mathbb{T}^3 \cong \mathbb{R}^3$ 对应于平移向量的得分, $T_RSO(3) \cong \mathbb{R}^3$ 对应于旋转向量的得分, $T_\theta SO(2)^m$ 对应于扭转角的得分. 置信度模型则输出配体姿态的置信度.

得分模型和置信度模型的架构类似, 基于 $SE(3)$ 等变卷积网络在点云上的应用. 然而, 得分模型在粗粒化的蛋白质表示上操作, 只考虑 α -碳原子, 而置信度模型可以访问全原子结构. 这种多尺度设置可以提供更好的性能, 并在原子尺度上显著加快处理速度.

结构被表示为由配体原子, 蛋白质残基和(对于置信度模型而言)蛋白质原子组成的异质几何图. 残基节点接收经过训练的蛋白质序列的语言模型嵌入作为初始特征. 节点之间的稀疏连接基于距离截断, 距离截断取决于要连接的节点类型和扩散时间.

为收集置信度模型 $d(x, y)$ 的训练数据, 运行经过训练的扩散模型, 为每个训练样本获取一组候选姿态, 并通过测试每个姿态的 RMSD 是否低于 $2\text{ \AA}$ 来生成标签. 然后, 使用交叉熵损失对置信度模型进行训练, 以正确预测每个姿态的二进制标签. 在推断过程中, 扩散模型并行生成 N 个姿态, 将它们传递给置信度模型, 根据其对姿态的 RMSD 是否低于 $2\text{ \AA}$ 的置信度对它们进行排名.

代码

位于 `./demo/mol_doc_pdb` 中, 包含训练脚本 `mol_doc_pdb_train.py` 和推理脚本

`mol_doc_pdb_infer.py`.

文件结构

```
|--dataset                                # 数据集文件
  |--pdbbind
    |--processed                          # 经过初步处理的数据
      |--PDBBind_processed               # 蛋白质受体和种子配体文件
        |--split                         # 将数据集划分为训练集和测试集的标签
          |--esm2_3billion_embeddings.pt # 语言模型esm的checkpoint
|--checkpoint                             # 检查点, 训练得到的模型
  |--pdbbind
    |--train                             # 训练过程中保存的checkpoint
    |--infer                             # 推理所用的经过训练的模型及参数配置checkpoint
      |--pdbbind.pth
|--gnn4s                                  # gnn4s库
  |--data                                # 数据类
    |--dataset.py
    |--load.py                           # 读取数据的函数
    |--process.py                         # 处理数据的函数
    |--molecule.py                      # 分子类
    |--protein.py                        # 蛋白质类
    |--noise.py                           # 添加噪声
  |--model                               # 网络模型
    |--gnn_tensor_product.py
    |--embedding.py
    |--mlp.py
  |--diffusion                           # 扩散过程
  |--train                               # 训练
    |--loss.py
    |--trainer.py
  |--evaluation                           # 评价
  |--postprocess                          # 后处理可视化
  |--utils
    |--logger.py                         # 记录日志
    |--setting.py                       # 文件和cudnn等设置
  |--__init__.py
|--config                                # 参数配置文件
  |--mol_doc_pdb_train.yaml              # 训练参数
  |--mol_doc_pdb_infer.yaml              # 推理参数
|--mol_doc_pdb_train.py                  # 训练脚本
|--mol_doc_pdb_infer.py                  # 推理脚本
```

主要流程

训练 在训练脚本 `mol_doc_pdb_train.py` 中, 首先读取参数设置, 并进行一些设置

```
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/mol_doc_pdb_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dir)
    logger.log_config(config)

    # Configuring file system resource sharing strategy
    gnn4s.utils.set_sharing_strategy('file_system')
    # Setting file descriptor limits in multi-process training
    gnn4s.utils.setrlimit(64000)
    # Configuring optimization options for the CuDNN library
    gnn4s.utils.use_cudnn_benchmark(config.cudnn_benchmark)
```

然后读取训练数据, 对训练数据进行一些处理, 转为图数据格式.

```
# dataset
load_fn = gnn4s.data.load_lm_embeddings
process_fn = gnn4s.data.process_complex
t_to_sigma = gnn4s.data.TimeToSigma(
    tr_sigma_min=config.diffusion.tr_sigma_min,
    tr_sigma_max=config.diffusion.tr_sigma_max,
    rot_sigma_min=config.diffusion.rot_sigma_min,
    rot_sigma_max=config.diffusion.rot_sigma_max,
    tor_sigma_min=config.diffusion.tor_sigma_min,
    tor_sigma_max=config.diffusion.tor_sigma_max)
transform_fn = gnn4s.data.NoiseTransform(t_to_sigma=t_to_sigma,
    no_torsion=config.diffusion.no_torsion, all_atom=config.data.all_atom)
train_dataset = gnn4s.data.PDBDataset(
    dirt=config.data.dirt, load_fn=load_fn, process_fn=process_fn,
    transform=transform_fn, cache_path=config.data.cache_dirt,
    split_path=config.data.split_dirt,
    esm_embd_path=config.data.esm_embd_path,
    complex_limit=config.data.complex_limit,
    worker_num=config.data.worker_num,
    lig_size_max=config.data.lig_size_max,
    popsize=config.data.matching_popsize,
    maxiter=config.data.matching_maxiter,
    matching=(not config.diffusion.no_torsion),
    keep_original=True,
    conformer_num=config.data.conformer_num,
    remove_hs=config.data.remove_hs,
    rec_cutoff_radius=config.data.rec_radius,
    ca_nb_num_max=config.data.ca_nb_num_max,
    all_atom=config.data.all_atom,
    atom_cutoff_radius=config.data.atom_radius,
    atom_nb_num_max=config.data.atom_nb_num_max)
```

搭建得分模型,

```
# model
timestep_embd = gnn4s.model.TimestepEmbedding(config.model.embd_type,
        config.model.sigma_embd_dim, config.model.embd_scale)
lm_embd_type = 'esm' if config.data.esm_embd_path is not None else None
score_model = gnn4s.model.TensorProductModel(
    t_to_sigma=t_to_sigma, timestep_embd_func=timestep_embd,
    ns=config.model.ns, nv=config.model.nv,
    sigma_embd_dim=config.model.sigma_embd_dim,
    dist_embd_dim=config.model.dist_embd_dim,
    cro_dist_embd_dim=config.model.cro_dist_embd_dim,
    lm_embd_type=lm_embd_type,
    lig_radius_max=config.model.max_radius,
    cro_dist_max=config.model.cro_dist_max,
    dropout_rate=config.model.dropout_rate,
    dynamic_cro_max=config.model.dynamic_cro_max,
    use_second_order_repr=config.model.use_second_order_repr,
    use_batch_norm=(not config.model.no_batch_norm),
    conv_layer_num=config.model.conv_layer_num,
    no_torsion=config.diffusion.no_torsion,
    scale_by_sigma=config.model.scale_by_sigma,
    conf_mode=False, device=config.device)
score_model = gnn4s.model.to_data_parallel(
    score_model, config.device, config.no_parallel)
gnn4s.model.count_parameters_number(score_model)
```

根据扩散过程构造损失函数

```
# loss
loss = gnn4s.train.LossScoreProduct(score_model, t_to_sigma,
        tra_weight=config.diffusion.tr_weight,
    rot_weight=config.diffusion.rot_weight,
        tor_weight=config.diffusion.tor_weight,
    no_torsion=config.diffusion.no_torsion)
```

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```
# train
optimizer = gnn4s.train.AdamOptimizer(score_model.parameters(),
    lr=config.train.lr)
scheduler = gnn4s.train.ExponentialLR(optimizer, gamma=0.999)
trainer = gnn4s.train.TrainerDiffDock(train_dataset, config.train.batch_size,
    config.data.worker_num, config.data.pin_memory,
    score_model, loss, optimizer, scheduler, config.train.epoch_num,
    use_ema=config.train.use_ema, device=config.device, logger=logger)
trainer.train()
```

以上是训练的整个流程.

推理 在推理脚本 `mol_doc_pdb_infer.py` 中, 首先读取参数设置, 加载checkpoint,

```
import gnn4s
```

```

def diffdock_inference(data_path, output_dirt, inference_sample_size):
    # configuration
    config = gnn4s.load_config('./config/mol_doc_pdb_infer.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)

    # load checkpoint
    ckpt_dict =
    gnn4s.utils.load_ckpt(f'{config.ckpt.dirt}/{config.ckpt.name}.pth')
    ckpt_config = ckpt_dict['config']
    logger.log_config(vars(ckpt_config))

```

然后读取测试数据, 对训练数据进行一些处理, 转为图数据格式, 并提取其中的测试数据

```

# dataset and dataloader
rec_path_list, lig_description = gnn4s.data.load_rec_lig_csv(data_path)
load_fn = gnn4s.data.load_lm_embeddings
process_fn = gnn4s.data.process_complex
test_dataset = gnn4s.data.PDBDataset(
    dirt='', load_fn=load_fn, process_fn=process_fn,
    cache_path=config.data.cache_dirt,
    esm_embd_path=config.data.esm_embd_dirt,
    worker_num=ckpt_config.data.worker_num,
    lig_description=lig_description, require_lig=True,
    popsize=ckpt_config.data.matching_popsize,
    maxiter=ckpt_config.data.matching_maxiter,
    matching=False, keep_original=False,
    remove_hs=ckpt_config.data.remove_hs,
    keep_local_structure=config.data.keep_local_structure,
    rec_path_list=rec_path_list,
    rec_cutoff_radius=ckpt_config.data.rec_radius,
    ca_nb_num_max=ckpt_config.data.ca_nb_num_max,
    all_atom=ckpt_config.data.all_atom_score,
    atom_cutoff_radius=ckpt_config.data.atom_radius,
    atom_nb_num_max=ckpt_config.data.atom_nb_num_max)
test_loader = gnn4s.data.DataLoader(
    dataset=test_dataset, batch_size=1, shuffle=False)

if 'confi_model_state_dict' in ckpt_dict:
    confi_test_dataset = gnn4s.data.PDBDataset(
        dirt=config.data.dirt, load_fn=load_fn, process_fn=process_fn,
        cache_path=config.data.cache_dirt,
        esm_embd_path=config.data.esm_embd_dirt,
        worker_num=ckpt_config.data.worker_num,
        lig_description=lig_description, require_lig=True,
        popsize=ckpt_config.data.matching_popsize,
        maxiter=ckpt_config.data.matching_maxiter,
        matching=False, keep_original=False,
        remove_hs=ckpt_config.data.remove_hs,
        rec_path_list=rec_path_list,
        rec_cutoff_radius=ckpt_config.data.rec_radius,
        ca_nb_num_max=ckpt_config.data.ca_nb_num_max,
        all_atom=ckpt_config.data.all_atom_confi,

```



```

        atom_cutoff_radius=ckpt_config.data.atom_radius,
        atom_nb_num_max=ckpt_config.data.atom_nb_num_max)
confi_complex_dict = {d.name: d for d in confi_test_dataset}

```

搭建得分模型和置信度模型, 并读取训练得到的模型参数,

```

# score model
csm = ckpt_config.score_model
t_to_sigma = gnn4s.data.TimeToSigma(
    ckpt_config.diffusion.tr_sigma_min, ckpt_config.diffusion.tr_sigma_max,
    ckpt_config.diffusion.rot_sigma_min, ckpt_config.diffusion.rot_sigma_max,
    ckpt_config.diffusion.tor_sigma_min, ckpt_config.diffusion.tor_sigma_max)
timestep_embd = gnn4s.model.TimeStepEmbedding(
    csm.embd_type, csm.sigma_embd_dim, csm.embd_scale)
score_model = gnn4s.model.TensorProductModel(
    t_to_sigma=t_to_sigma, timestep_embd_func=timestep_embd,
    ns=csm.ns, nv=csm.nv, sigma_embd_dim=csm.sigma_embd_dim,
    dist_embd_dim=csm.dist_embd_dim,
    cro_dist_embd_dim=csm.cro_dist_embd_dim,
    lm_embd_type=csm.lm_embd_type, lig_radius_max=csm.max_radius,
    cro_dist_max=csm.cro_dist_max, dropout_rate=csm.dropout_rate,
    dynamic_cro_max=csm.dynamic_cro_max,
    use_second_order_repr=csm.use_second_order_repr,
    use_batch_norm=(not csm.no_batch_norm),
    conv_layer_num=csm.conv_layer_num,
    no_torsion=ckpt_config.diffusion.no_torsion,
    scale_by_sigma=csm.scale_by_sigma,
    conf_mode=False, device=config.device)
score_model = gnn4s.model.load_state_dict(
    score_model, ckpt_dict['score_model_state_dict'],
    'eval', config.device, config.no_parallel)

```

构造采样器, 将模型和扩散过程传入采样器, 用采样器生成配体的候选姿态, 并使用置信度模型计算置信度.

```

# sampler
transform = gnn4s.data.NoiseTransform(
    t_to_sigma=t_to_sigma, no_torsion=False, all_atom=False)
sampler = gnn4s.diffusion.Sampler(
    t_to_sigma=t_to_sigma, transform=transform,
    score_model=score_model, confi_model=confi_model,
    no_torsion=ckpt_config.diffusion.no_torsion,
    score_all_atom=ckpt_config.data.all_atom_score,
    confi_all_atom=ckpt_config.data.all_atom_confi,
    tr_sigma_min=ckpt_config.diffusion.tr_sigma_min,
    tr_sigma_max=ckpt_config.diffusion.tr_sigma_max,
    rot_sigma_min=ckpt_config.diffusion.rot_sigma_min,
    rot_sigma_max=ckpt_config.diffusion.rot_sigma_max,
    tor_sigma_min=ckpt_config.diffusion.tor_sigma_min,
    tor_sigma_max=ckpt_config.diffusion.tor_sigma_max,
    no_random=ckpt_config.diffusion.no_random,
    use_ode=ckpt_config.diffusion.use_ode,
    no_final_step_noise=ckpt_config.diffusion.no_final_step_noise,
    device=config.device)

```



```
...
```

最后进行后处理, 保存生成的配体和置信度

```
# save generated ligand
lig = orig_complex_graph.mol[0]
gnn4s.postprocess.write_mol_with_cord(
    lig, lig_pos, ckpt_config.data.remove_hs, confidence,
    output_dirt, idx, data_list[0]["name"][0])
```

以上是推理的整个流程.

参数设置

训练和推理脚本中的相关部分:

```
import gnn4s

def main():
    # configuration
    config = gnn4s.load_config('./config/flag_simple_train.yaml')

    # logger
    logger = gnn4s.utils.Logger(config.log.dirt)
    logger.log_config(config)
    ...
```

训练时还需要设置多进程之间的共享策略(这里使用文件系统), 处理数据时同时打开的文件数目上限(这里设置为64000), 以及CuDNN的优化选项(这里使用cuDNN的自动调优).

```
# Configuring file system resource sharing strategy
gnn4s.utils.set_sharing_strategy('file_system')
# Setting file descriptor limits in multi-process training
gnn4s.utils.setrlimit(64000)
# Configuring optimization options for the CuDNN library
gnn4s.utils.use_cudnn_benchmark(config.cudnn_benchmark)
...
```

推理时还需要读取checkpoint

```
# load checkpoint
ckpt_path = f'{config.ckpt.dirt}/{config.ckpt.name}.pth'
ckpt_dict = torch.load(ckpt_path)
ckpt_config = ckpt_dict['config']
logger.log_config(vars(ckpt_config))
...
```

训练所用的参数设置存放在 ./config/mol_doc_pdb_train.yaml 中:

```
data:
    # name of dataset
    name: pddbbind
    # directory for storing dataset
```

```

dirt: ./dataset/pdbbind/processed/PDBBind_processed/
# directory storing index for splitting dataset
split_dirt: ./dataset/pdbbind/processed/splits/timesplit_no_lig_overlap_train
# directory for storing cached dataset
cache_dirt: ./dataset/pdbbind/processed/cache
# path for storing LM embeddings for the receptor features
esm_embd_path: ./dataset/pdbbind/processed/esm2_3billion_embeddings.pt

# whether or not use pin memory
pin_memory: False
# the number of training and validation complexes is capped
complex_limit: 0
# number of workers for preprocessing
worker_num: 1
# whether to use the all atoms model
all_atom: False
# cutoff on distances for receptor edges
rec_radius: 15
# maximum number of neighbors for each residue
ca_nb_num_max: 24
# cutoff on distances for atom connections
atom_radius: 5
# maximum number of atom neighbours for receptor
atom_nb_num_max: 8
# whether to remove HS
remove_hs: True
# number of conformers to match to each ligand
conformer_num: 1
# keeps the local structure when specifying an input with 3D coordinates
instead of generating them with RDKit
keep_local_structure: False
# number of samples to generate
sample_per_complex: 5

model:
# number of interaction layers
conv_layer_num: 1
# radius cutoff for geometric graph
max_radius: 5.0
# whether to normalise the score
scale_by_sigma: True
# number of hidden features per node of order 0
ns: 48
# number of hidden features per node of order >0
nv: 10
# embedding size for the distance
dist_embd_dim: 64
# embeddings size for the cross distance
cro_dist_embd_dim: 64
# If set, it removes the batch norm
no_batch_norm: False
# whether to use only up to first order representations or also second
use_second_order_repr: False
# maximum cross distance in case not dynamic
cro_dist_max: 80
# whether to use the dynamic distance cutoff

```

```
dynamic_cro_max: True
# mlp dropout rate
dropout_rate: 0.1
# type of diffusion time embedding
embd_type: sinusoidal
# type of language model embedding
lm_embd_type: esm
# size of the embedding of the diffusion time
sigma_embd_dim: 64
# parameter of the diffusion time embedding
embd_scale: 10000

diffusion:
# weight of translation loss
tr_weight: 0.33
# weight of rotation loss
rot_weight: 0.33
# weight of torsional loss
tor_weight: 0.33
# minimum sigma for rotational component
rot_sigma_min: 0.03
# maximum sigma for rotational component
rot_sigma_max: 1.55
# minimum sigma for translational component
tr_sigma_min: 0.1
# maximum sigma for translational component
tr_sigma_max: 19.0
# minimum sigma for torsional component
tor_sigma_min: 0.0314
# maximum sigma for torsional component
tor_sigma_max: 3.14
# if set only rigid matching
no_torsion: False
# use no randomness in reverse diffusion
no_random: False
# use no noise in the final step of the reverse diffusion
no_final_step_noise: False
# use ODE formulation for inference
use_ode: False

train:
# batch size in training
batch_size: 16
# learning rate
lr: 0.001
# number of training epoch
epoch_num: 100
# number of warmup step
warmup_step: 0
# whether use exponential moving average
use_ema: True

log:
# directory for saving log
dirt: ./log/pdbbind
```

```
# computing device
device: cuda
# whether or not use parallel training
no_parallel: False
# CUDA optimization parameter for faster training
cudnn_benchmark: True
```

推理所用的参数设置存放在 `./config/mol_doc_pdb_infer.yaml` 中

```
data:
  # name of dataset
  name: pdbbind
  # directory for storing dataset
  dirt: ''
  # path for storing dataset
  path: ./dataset/pdbbind/inference/protein_ligand.csv
  # directory for storing cached dataset
  cache_dirt: ./dataset/pdbbind/processed/cache
  # directory for storing output file
  output_dirt: ./dataset/pdbbind/output/diffdock
  # path for storing LM embeddings for the receptor features
  esm_embd_dirt: ./dataset/pdbbind/processed/esm2_output

  # keeps the local structure when specifying an input with
  # 3D coordinates instead of generating them with RDKit
  keep_local_structure: False
  # number of samples to generate
  sample_per_complex: 5

ckpt:
  # name of checkpoint
  name: pdbbind
  # directory for storing checkpoint
  dirt: ./checkpoint/pdbbind/infer

inference:
  sample_size: 6
  # number of denoising steps
  step_num: 10

log:
  # directory for saving log
  dirt: ./log/pdbbind

# computing device
device: cuda
# whether or not use parallel training
no_parallel: True
```

推理所用的checkpoint存放在 `./checkpoint/pdbbind/infer/pdbbind.pth` 中, 包含训练得到的模型参数以及相关的一些参数设置

```
{
  'config': {'data': {}, 'score_model': {}, 'confi_model': {}, 'diffusion': {}}
  'score_model_state_dict': {}
  'confi_model_state_dict': {}
}
```

读取数据

训练脚本中的相关部分:

```
def main():
    ...
    # dataset
    load_fn = gnn4s.data.load_lm_embeddings
    process_fn = gnn4s.data.process_complex
    t_to_sigma = gnn4s.data.TimeToSigma(
        tr_sigma_min=config.diffusion.tr_sigma_min,
        tr_sigma_max=config.diffusion.tr_sigma_max,
        rot_sigma_min=config.diffusion.rot_sigma_min,
        rot_sigma_max=config.diffusion.rot_sigma_max,
        tor_sigma_min=config.diffusion.tor_sigma_min,
        tor_sigma_max=config.diffusion.tor_sigma_max)
    transform_fn = gnn4s.data.NoiseTransform(t_to_sigma=t_to_sigma,
        no_torsion=config.diffusion.no_torsion, all_atom=config.data.all_atom)
    train_dataset = gnn4s.data.PDBDataset(
        dirt=config.data.dirt, load_fn=load_fn, process_fn=process_fn,
        transform=transform_fn, cache_path=config.data.cache_dirt,
        split_path=config.data.split_dirt,
        esm_embd_path=config.data.esm_embd_path,
        complex_limit=config.data.complex_limit,
        worker_num=config.data.worker_num,
        lig_size_max=config.data.lig_size_max,
        popsize=config.data.matching_popsize,
        maxiter=config.data.matching_maxiter,
        matching=(not config.diffusion.no_torsion),
        keep_original=True,
        conformer_num=config.data.conformer_num,
        remove_hs=config.data.remove_hs,
        rec_cutoff_radius=config.data.rec_radius,
        ca_nb_num_max=config.data.ca_nb_num_max,
        all_atom=config.data.all_atom,
        atom_cutoff_radius=config.data.atom_radius,
        atom_nb_num_max=config.data.atom_nb_num_max)
    ...
```

推理脚本中的相关部分:

```
def main():
    ...
    # dataset and data loader
    rec_path_list, lig_description = gnn4s.data.load_rec_lig_csv(data_path)
    load_fn = gnn4s.data.load_lm_embeddings
    process_fn = gnn4s.data.process_complex
    test_dataset = gnn4s.data.PDBDataset(
```

```

dirt='', load_fn=load_fn, process_fn=process_fn,
cache_path=config.data.cache_dirt,
esm_embd_path=config.data.esm_embd_dirt,
worker_num=ckpt_config.data.worker_num,
lig_description=lig_description, require_lig=True,
popsize=ckpt_config.data.matching_popsize,
maxiter=ckpt_config.data.matching_maxiter,
matching=False, keep_original=False,
remove_hs=ckpt_config.data.remove_hs,
keep_local_structure=config.data.keep_local_structure,
rec_path_list=rec_path_list,
rec_cutoff_radius=ckpt_config.data.rec_radius,
ca_nb_num_max=ckpt_config.data.ca_nb_num_max,
all_atom=ckpt_config.data.all_atom_score,
atom_cutoff_radius=ckpt_config.data.atom_radius,
atom_nb_num_max=ckpt_config.data.atom_nb_num_max)
test_loader = gnn4s.data.DataLoader(
    dataset=test_dataset, batch_size=1, shuffle=False)

if 'confi_model_state_dict' in ckpt_dict:
    confi_test_dataset = gnn4s.data.PDBDataset(
        dirt=config.data.dirt, load_fn=load_fn, process_fn=process_fn,
        cache_path=config.data.cache_dirt,
        esm_embd_path=config.data.esm_embd_dirt,
        worker_num=ckpt_config.data.worker_num,
        lig_description=lig_description, require_lig=True,
        popsize=ckpt_config.data.matching_popsize,
        maxiter=ckpt_config.data.matching_maxiter,
        matching=False, keep_original=False,
        remove_hs=ckpt_config.data.remove_hs,
        rec_path_list=rec_path_list,
        rec_cutoff_radius=ckpt_config.data.rec_radius,
        ca_nb_num_max=ckpt_config.data.ca_nb_num_max,
        all_atom=ckpt_config.data.all_atom_confi,
        atom_cutoff_radius=ckpt_config.data.atom_radius,
        atom_nb_num_max=ckpt_config.data.atom_nb_num_max)
    confi_complex_dict = {d.name: d for d in confi_test_dataset}

...

```

`gnn4s.data.load_lm_embeddings` 位于 `./gnn4s/data/load.py` 中, 读取蛋白质受体和配体种子构象信息.

`gnn4s.data.process_complex` 位于 `./gnn4s/data/process.py` 中, 将受体和配体信息转化为 PyGHeteroData 异构图.

`gnn4s.data.TimeToSigma` 位于 `./gnn4s/data/noise.py` 中, 根据扩散过程中的时间步计算添加噪声所服从的先验分布的标准差. 这里计算标准差的方式为

$$\sigma_{tr}(t) = (\sigma_{tr}^{min})^{1-t} (\sigma_{tr}^{max})^t.$$

```

class TimeToSigma():
    def __init__(self, tr_sigma_min: float, tr_sigma_max: float,
                 rot_sigma_min: float, rot_sigma_max: float,
                 tor_sigma_min: float, tor_sigma_max: float) -> None:
        """ Standard deviation of the prior distribution.
        Args:

```

```

        tr_sigma_min: minimum value of the standard deviation related to
translation.
        tr_sigma_max: maximum value of the standard deviation related to
translation.
        rot_sigma_min: minimum value of the standard deviation related to
rotation.
        rot_sigma_max: maximum value of the standard deviation related to
rotation.
        tor_sigma_min: minimum value of the standard deviation related to
torsion.
        tor_sigma_max: maximum value of the standard deviation related to
torsion.
    """
    ...

def __call__(self, t_tr, t_rot, t_tor):
    """ Calculate the standard deviation of the prior distribution.
    Args:
        t_tr, t_rot, t_tor: time
    """
    tr_sigma = self.tr_sigma_min ** (1-t_tr) * self.tr_sigma_max ** t_tr
    rot_sigma = self.rot_sigma_min ** (1-t_rot) * self.rot_sigma_max ** t_rot
    tor_sigma = self.tor_sigma_min ** (1-t_tor) * self.tor_sigma_max ** t_tor
    return tr_sigma, rot_sigma, tor_sigma

```

`gnn4s.data.NoiseTransform` 位于 `./gnn4s/data/noise.py` 中, 在扩散过程中逐步将噪声添加到数据中.

```

class NoiseTransform(BaseTransform):
    def __init__(self, t_to_sigma, no_torsion, all_atom):
        """ Add noise to data.
        Args:
            t_to_sigma: function for calculating the standard deviation of
the prior distribution.
            no_torsion: whether to consider torsion
            all_atom: whether to consider all atom
        """
        ...

    def __call__(self, data):
        """ Add noise to data.
        Args:
            data: heterogeneous graph with ligand information
        """
        t = np.random.uniform()
        t_tr, t_rot, t_tor = t, t, t
        return self.apply_noise(data, t_tr, t_rot, t_tor)

    def apply_noise(self, data, t_tr, t_rot, t_tor,
                    tr_update=None, rot_update=None, torsion_update=None):
        """ Add noise to data.
        Args:
            data: heterogeneous graph with ligand information
            t_tr, t_rot, t_tor: time
            tr_update: update of the translation

```

```

        rot_update: update of the rotation angle
        torsion_update: update of the torsion angle
    """
    if not torch.is_tensor(data['ligand'].pos):
        data['ligand'].pos = random.choice(data['ligand'].pos)

    # Calculate the standard deviation of the prior distribution.
    tr_sigma, rot_sigma, tor_sigma = self.t_to_sigma(t_tr, t_rot, t_tor)
    self.set_time(data, t_tr, t_rot, t_tor, 1, self.all_atom, device=None)

    # Sampling update magnitude from the distribution.
    tr_update = torch.normal(mean=0, std=tr_sigma, size=(1, 3)
        ) if tr_update is None else tr_update
    rot_update = sample_vec(eps=rot_sigma
        ) if rot_update is None else rot_update
    torsion_update = np.random.normal(
        loc=0.0, scale=tor_sigma, size=data['ligand'].edge_mask.sum()
        ) if torsion_update is None else torsion_update
    torsion_update = None if self.no_torsion else torsion_update

    # Modify the conformation of the ligand.
    self.modify_conformer(data, tr_update,
        torch.from_numpy(rot_update).float(), torsion_update)

    # Calculate score.
    data.tr_score = -tr_update / tr_sigma ** 2
    data.rot_score = torch.from_numpy(
        score_vec(vec=rot_update, eps=rot_sigma)).float().unsqueeze(0)
    data.tor_score = None if self.no_torsion else
        torch.from_numpy(score(torsion_update, tor_sigma)).float()
    data.tor_sigma_edge = None if self.no_torsion else
        np.ones(data['ligand'].edge_mask.sum()) * tor_sigma
    return data
...
def modify_conformer(self, data, tr_update, rot_update, torsion_update):
    """ Modify the conformation of the ligand according to the update.
    Args:
        data: heterogeneous graph with ligand information
        tr_update: update of the translation
        rot_update: update of the rotation angle
        torsion_update: update of the torsion angle
    """

    # Translate and rotate the ligand.
    lig_center = torch.mean(data['ligand'].pos, dim=0, keepdim=True)
    rot_mat = self.axis_angle_to_matrix(rot_update.squeeze())
    rigid_new_pos = (data['ligand'].pos - lig_center
        ) @ rot_mat.T + tr_update + lig_center

    # Twist the rotatable bonds in the molecule.
    if torsion_update is not None:
        flexible_new_pos = self.modify_conformer_torsion_angles(
            rigid_new_pos,
            data['ligand', 'ligand'].edge_index.T[data['ligand'].edge_mask],
            data['ligand'].mask_rotate if isinstance(
                data['ligand'].mask_rotate, np.ndarray
            ) else data['ligand'].mask_rotate[0],

```



```

        torsion_update
    ).to(rigid_new_pos.device)
    R, t = self.rigid_transform_Kabsch_3D_torch(
        flexible_new_pos.T, rigid_new_pos.T)
    aligned_flexible_pos = flexible_new_pos @ R.T + t.T
    data['ligand'].pos = aligned_flexible_pos
else:
    data['ligand'].pos = rigid_new_pos
return data
...

```

数据类 `gnn4s.data.PDBDataset` 位于 `./gnn4s/data/dataset.py` 中.

```

class PDBDataset(PyGDataset):
    def __init__(self, dirt, load_fn, process_fn, transform=None,
        cache_path: str='data_files/processed/cache',
        split_path: str='data_files/processed/',
        esm_embd_path: str=None, complex_limit: int=0, worker_num:
int=1,

        lig_description=None, require_lig: bool=False,
        lig_size_max: int=None, popsize: int=15, maxiter: int=15,
        matching: bool=True, keep_original: bool=False,
        conformer_num: int=1,
        remove_hs: bool=False, keep_local_structure: bool=False,
        rec_path_list=None, rec_cutoff_radius: float=30.0,
        ca_nb_num_max: int=None, all_atom: bool=False,
        atom_cutoff_radius: float=5.0, atom_nb_num_max: int=None):
        super(PDBDataset, self).__init__(dirt, transform)
        """ PDBBind dataset
        Args:
            dirt: directory for storing dataset
            load_fn: function for loading the dataset
            process_fn: function for processing the dataset
            transform: noise transformation function
            cache_path: directory for storing cached dataset
            split_dirt: directory storing index for splitting dataset
            esm_embd_path: path for storing LM embeddings for the receptor

        features
        capped

            complex_limit: the number of training and validation complexes is
            capped

            worker_num: number of workers for preprocessing
            lig_description: description of ligand
            require_lig: whether to require ligand
            lig_size_max: maximum number of heavy atoms in ligand
            popsize: differential evolution popsize parameter in matching
            maxiter: differential evolution maxiter parameter in matching
            matching: whether to matching
            keep_original: whether to keep the original structure
            conformer_num: number of conformers to match to each ligand
            remove_hs: whether to remove Hs
            keep_local_structure: keeps the local structure when specifying an
                input with 3D coordinates instead of generating them with RDKit
            rec_path_list: list of path of receptor
            rec_cutoff_radius: cutoff on distances for receptor edges
            ca_nb_num_max: maximum number of neighbors for each residue

```

```

        all_atom: whether to use the all atoms model
        atom_cutoff_radius: cutoff on distances for atom connections
        atom_nb_num_max: maximum number of atom neighbours for receptor
    """
    ...
    # Pre-process data
    if self.rec_path_list is None or self.lig_description is None:
        self.pre_process()
    else:
        self.pre_process_inf()
    ...

```

搭建网络

训练脚本中的相关部分:

```

def main():
    ...
    # model
    timestep_embd = gnn4s.model.TimestepEmbedding(config.model.embd_type,
                                                    config.model.sigma_embd_dim, config.model.embd_scale)
    lm_embd_type = 'esm' if config.data.esm_embd_path is not None else None
    score_model = gnn4s.model.TensorProductModel(
        t_to_sigma=t_to_sigma, timestep_embd_func=timestep_embd,
        ns=config.model.ns, nv=config.model.nv,
        sigma_embd_dim=config.model.sigma_embd_dim,
        dist_embd_dim=config.model.dist_embd_dim,
        cro_dist_embd_dim=config.model.cro_dist_embd_dim,
        lm_embd_type=lm_embd_type,
        lig_radius_max=config.model.max_radius,
        cro_dist_max=config.model.cro_dist_max,
        dropout_rate=config.model.dropout_rate,
        dynamic_cro_max=config.model.dynamic_cro_max,
        use_second_order_repr=config.model.use_second_order_repr,
        use_batch_norm=(not config.model.no_batch_norm),
        conv_layer_num=config.model.conv_layer_num,
        no_torsion=config.diffusion.no_torsion,
        scale_by_sigma=config.model.scale_by_sigma,
        conf_mode=False, device=config.device)
    score_model = gnn4s.model.to_data_parallel(
        score_model, config.device, config.no_parallel)
    gnn4s.model.count_parameters_number(score_model)
    ...

```

推理时调用 `gnn4s.model.load_state_dict` 读取经过训练的模型.

模型位于 `./gnn4s/model.gnn_tensor_product.py` 中

```

class TensorProductModel(nn.Module):
    def __init__(self, t_to_sigma, timestep_embd_func, ns: int=16, nv: int=4,
                  lig_edge_in_feat: int=4, sigma_embd_dim: int=32,
                  dist_embd_dim: int=32, cro_dist_embd_dim: int=32,
                  lm_embd_type=None, lig_radius_max: float=5.0,
                  rec_radius_max: float=30.0, cro_dist_max: float=250.0,
                  cen_dist_max: float=30, dropout_rate=0.0,

```

```

        dynamic_cro_max: bool=False,
        use_second_order_repr=False, sh_lmax: int=2,
        use_batch_norm: bool=True, conv_layer_num: int=2,
        no_torsion: bool=False, scale_by_sigma: bool=True,
        conf_mode: bool=False, conf_dropout_rate: float=0.0,
        conf_no_batch_norm: bool=False,
        conf_output_num: int=1, device: str='cpu') -> None:
super(TensorProductModel, self).__init__()
""" Tensor product model.
Args:
    t_to_sigma:
    timestep_embd_func: time embedding function
    ns: number of hidden features per node of order 0
    nv: number of hidden features per node of order >0
    lig_edge_in_feat:
    sigma_embd_dim: size of the embedding of the diffusion time
    dist_embd_dim: embedding size for the distance
    cro_dist_embd_dim: embeddings size for the cross distance
    lm_embd_type: type of language model embedding
    lig_radius_max: cutoff radius for ligand graph
    rec_radius_max: cutoff radius for receptor graph
    cro_dist_max: maximum cross distance in case not dynamic
    cen_dist_max: maximum center distance in case not dynamic
    dropout_rate: mlp dropout rate
    dynamic_cro_max: whether to use the dynamic distance cutoff
    use_second_order_repr: whether to use only up to first order
        representations or also second
    sh_lmax: maximum spherical harmonic degree
    use_batch_norm: whether to use the batch norm
    conv_layer_num: number of interaction layers
    no_torsion: whether to consider torsion
    scale_by_sigma: whether to normalize the score
    conf_mode: whether the model is a confidence model
    conf_dropout_rate: mlp dropout rate
    conf_no_batch_norm: whether to remove the batch norm
    conf_output_num: number of output
    device: computing device
"""
...
def forward(self, data):
    """ Forward propagation.
    Args:
        data: heterogeneous graph containing receptor and ligand information.
    """
    ...
    # build ligand graph
    lig_node_attr, lig_edge_idx, lig_edge_attr, lig_edge_sh =
        self.build_lig_conv_graph(data)
    ...
    # build receptor graph
    rec_node_attr, rec_edge_idx, rec_edge_attr, rec_edge_sh =
        self.build_rec_conv_graph(data)
    ...
    # build cross graph
    cro_edge_idx, cro_edge_attr, cro_edge_sh =
        self.build_cross_conv_graph(data, cross_cutoff)

```

```

...
for i in range(len(self.lig_conv_layer)):
    # intra graph message passing
    ...
    lig_intra_update = self.lig_conv_layer[i](
        lig_node_attr, lig_edge_idx, lig_edge_attr_, lig_edge_sh)

    # inter graph message passing
    ...
    lig_inter_update = self.rec_to_lig_conv_layer[i](
        rec_node_attr, cro_edge_idx, rec_to_lig_edge_attr_, cro_edge_sh,
        node_out_dim=lig_node_attr.shape[0])

    if i != len(self.lig_conv_layer) - 1:
        ...
        rec_intra_update = self.rec_conv_layer[i](
            rec_node_attr, rec_edge_idx, rec_edge_attr_, rec_edge_sh)
        ...
        rec_inter_update = self.lig_to_rec_conv_layer[i](
            lig_node_attr, torch.flip(cro_edge_idx, dims=[0]),
            lig_to_rec_edge_attr_,
            cro_edge_sh, node_out_dim=rec_node_attr.shape[0])
        ...
        # update features with residual updates
        lig_node_attr = lig_node_attr + lig_intra_update + lig_inter_update

        if i != len(self.lig_conv_layer) - 1:
            ...
            rec_node_attr = rec_node_attr + rec_intra_update +
rec_inter_update

    # compute confidence score
    if self.conf_mode:
        ...
        confidence = self.conf_predictor(
            scatter_mean(scalar_lig_attr, data['ligand'].batch, dim=0)
        ).squeeze(dim=-1)
        return confidence

    # compute translational and rotational score vectors
    center_edge_idx, center_edge_attr, center_edge_sh =
        self.build_center_conv_graph(data)
    center_edge_attr = self.cen_edge_embd(center_edge_attr)
    ...
    global_pred = self.final_conv(
        lig_node_attr, center_edge_idx, center_edge_attr, center_edge_sh,
        node_out_dim=data.num_graphs)

    tr_pred = global_pred[:, :3] + global_pred[:, 6:9]
    rot_pred = global_pred[:, 3:6] + global_pred[:, 9:]
    ...
    # torsional components
    ...
    tor_pred = self.tor_bond_conv(
        lig_node_attr, tor_edge_idx, tor_edge_attr, tor_edge_sh,

```

```

        node_out_dim=data['ligand'].edge_mask.sum(), reduce='mean')
    tor_pred = self.tor_final_layer(tor_pred).squeeze(1)
    ...
    return tr_pred, rot_pred, tor_pred

```

损失函数

训练脚本中的相关部分

```

def main():
    ...
    # loss
    loss = gnn4s.train.LossScoreProduct(score_model, t_to_sigma,
        tra_weight=config.diffusion.tr_weight,
        rot_weight=config.diffusion.rot_weight,
        tor_weight=config.diffusion.tor_weight,
        no_torsion=config.diffusion.no_torsion)
    ...

```

`gnn4s.train.LossScoreProduct` 位于 `./gnn4s/train/loss.py` 中, 计算损失函数的值.

```

class LossScoreProduct():
    def __init__(self, model, t_to_sigma, tra_weight: float=1.0,
        rot_weight: float=1.0, tor_weight: float=1.0,
        no_torsion: bool=False) -> None:
        """ Loss function related to the score in product space.
        Args:
            model: model
            t_to_sigma: function for calculate the standard deviation
                of the prior distribution according to the time
            tra_weight: weight realated to translation term
            rot_weight: weight realated to rotation term
            tor_weight: weight realated to torsion term
            no_torsion: whether to consider torsion
        """
        ...
    def __call__(self, data):
        """ Calculate the value of loss function.
        Args:
            data: heterogeneous graph containing receptor and ligand information.
        """
        # predict the score
        tra_pred, rot_pred, tor_pred = self.model(data)

        # calculate the the standard deviation of the prior distribution
        # according to the time
        device = tra_pred.device
        tra_sigma, rot_sigma, tor_sigma = self.t_to_sigma(
            *[torch.cat([d.complex_t[noise_type] for d in data]
                ) if device.type == 'cuda' else data.complex_t[noise_type]
                for noise_type in ['tr', 'rot', 'tor']])
        mean_dims = (0, 1)

```

```

# translation component
tra_score = torch.cat([d.tr_score for d in data], dim=0
    ) if device.type == 'cuda' else data.tr_score
tra_sigma = tra_sigma.unsqueeze(-1)
tra_loss = ((tra_pred.cpu() - tra_score) ** 2 * tra_sigma ** 2
    ).mean(dim=mean_dims)

# rotation component
rot_score = torch.cat([d.rot_score for d in data], dim=0
    ) if device.type == 'cuda' else data.rot_score
rot_score_norm = gnn4s.data.so3.score_norm(rot_sigma.cpu()).unsqueeze(-1)
rot_loss = (((rot_pred.cpu() - rot_score) / rot_score_norm) ** 2
    ).mean(dim=mean_dims)

# torsion component
if not self.no_torsion:
    edge_tor_sigma = torch.from_numpy(
        np.concatenate([d.tor_sigma_edge for d in data]
            if device.type == 'cuda' else data.tor_sigma_edge))
    tor_score = torch.cat([d.tor_score for d in data], dim=0
        ) if device.type == 'cuda' else data.tor_score
    tor_score_norm = torch.tensor(gnn4s.data.torus.score_norm(
        edge_tor_sigma.cpu().numpy())).float()
    tor_loss = ((tor_pred.cpu() - tor_score) ** 2 / tor_score_norm)
    tor_loss = tor_loss.mean() * torch.ones(1, dtype=torch.float)
else:
    tor_loss = torch.zeros(1, dtype=torch.float)

loss = self.tra_weight*tra_loss + self.rot_weight*rot_loss +
    self.tor_weight*tor_loss
return loss

```

训练网络

最后将训练数据, 网络和损失函数传入训练器, 训练网络.

```

def main():
    ...
    # train
    optimizer = gnn4s.train.AdamOptimizer(score_model.parameters(),
lr=config.train.lr)
    scheduler = gnn4s.train.ExponentialLR(optimizer, gamma=0.999)
    trainer = gnn4s.train.TrainerDiffDock(train_dataset, config.train.batch_size,
        config.data.worker_num, config.data.pin_memory,
        score_model, loss, optimizer, scheduler, config.train.epoch_num,
        use_ema=config.train.use_ema, device=config.device, logger=logger)
    trainer.train()
    ...

```

`gnn4s.train.TrainerDiffDock` 位于 `./gnn4s/trian/trainer.py` 中.

```

class TrainerDiffDock():
    def __init__(self, dataset, batch_size: int=16, worker_num: int=1,
        pin_memory: bool=False, model=None, loss=None,

```

```

        optimizer=None, scheduler=None, epoch_num: int=10000,
        warmup_step: int=0, grad_clip: float=0.0,
        ema_rate: float=0.9999, use_ema: bool=False,
        save_interval: int=1, device='cpu', logger=None) -> None:
    """ Trainer for training model.
    Args:
        dataset: dataset
        batch_size: batch size
        worker_num: number of worker
        pin_memory: whether to use pin memory
        model: model
        loss: loss function
        optimizer: optimizer for minimizing the loss function
        scheduler: learning rate scheduler
        epoch_num: number of epochs
        warmup_step: number of warmup steps for learning rate
        grad_clip: maximum norm of gradients
        ema_rate: smoothing coefficient
        use_ema: whether to use Exponential Moving Average
        save_interval: interval for saving the model
        device: computing device
        logger: logger
    """
    ...

def step(self, batch):
    """ Running one step of training.
    Returns:
        loss: The average loss value of this state.
    """
    ...

def train(self):
    """ Train model. """
    ...
    # loop
    while self.current_epoch < self.epoch_num:
        i = 0
        for batch in self.loader:
            # train neural networks
            loss = self.step(batch)

            # print
            if loss is not None:
                info = (f'epoch: {self.current_epoch} | batch: {i} | '+'
                        f'loss: {loss.item():.3e} | time: {time.time()-
                        start_time:.2f}')
                self.logger.log_info(info)

            i += 1
            start_time = time.time()

        # schedule learning rate
        if self.scheduler is not None:
            self.scheduler.step()

```

```
self.current_epoch += 1  
...
```